

MODULE III

WINDOW TO VIEWPORT TRANSFORMATION

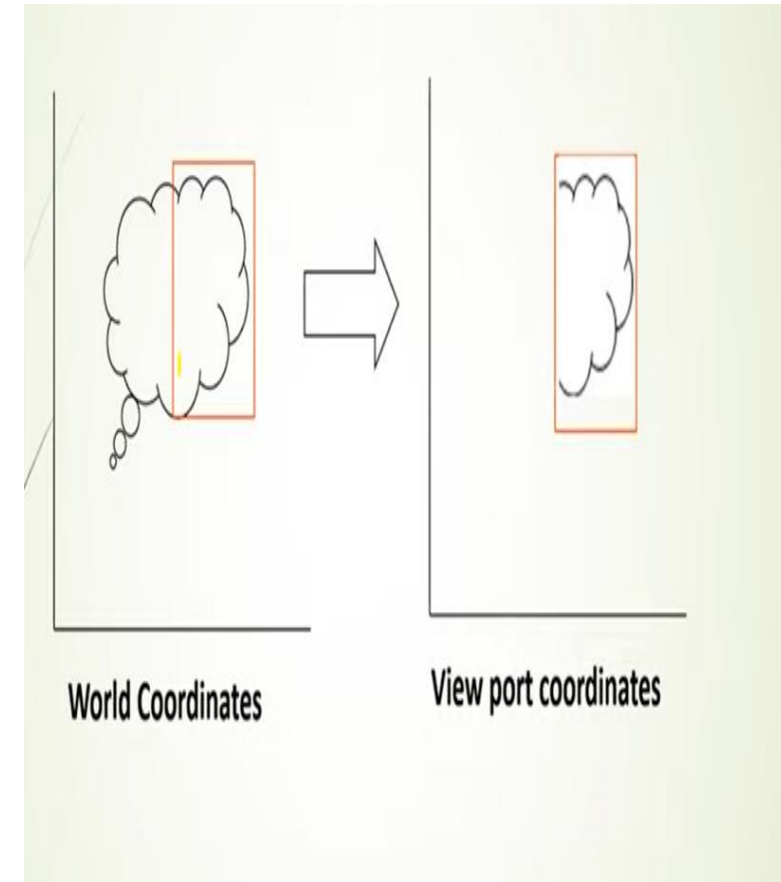
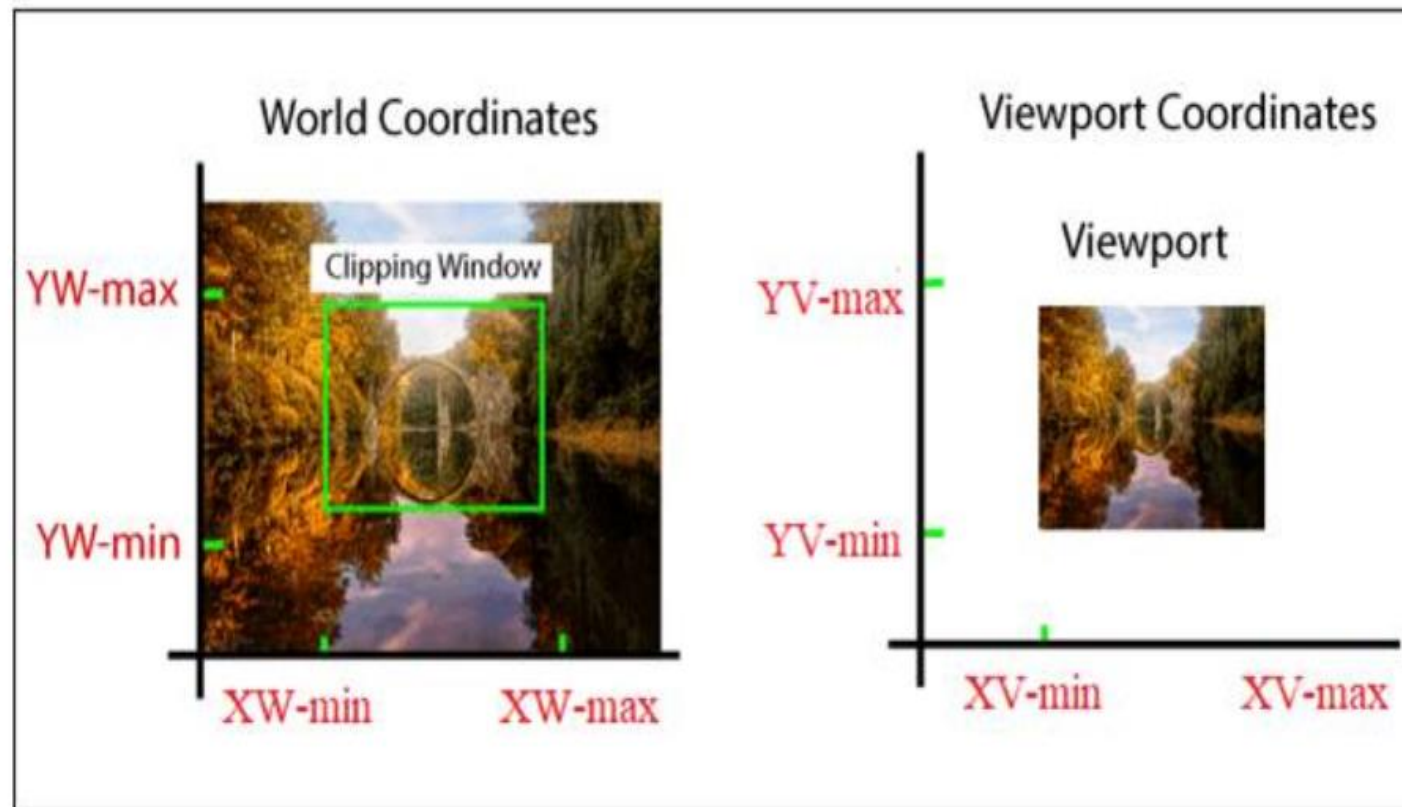
What is Window?

Window is a world coordinate area selected for display. So, the Window defines **what is to be viewed**. First of all, we have to decide which portion of the real world, we want to display.



View-Port

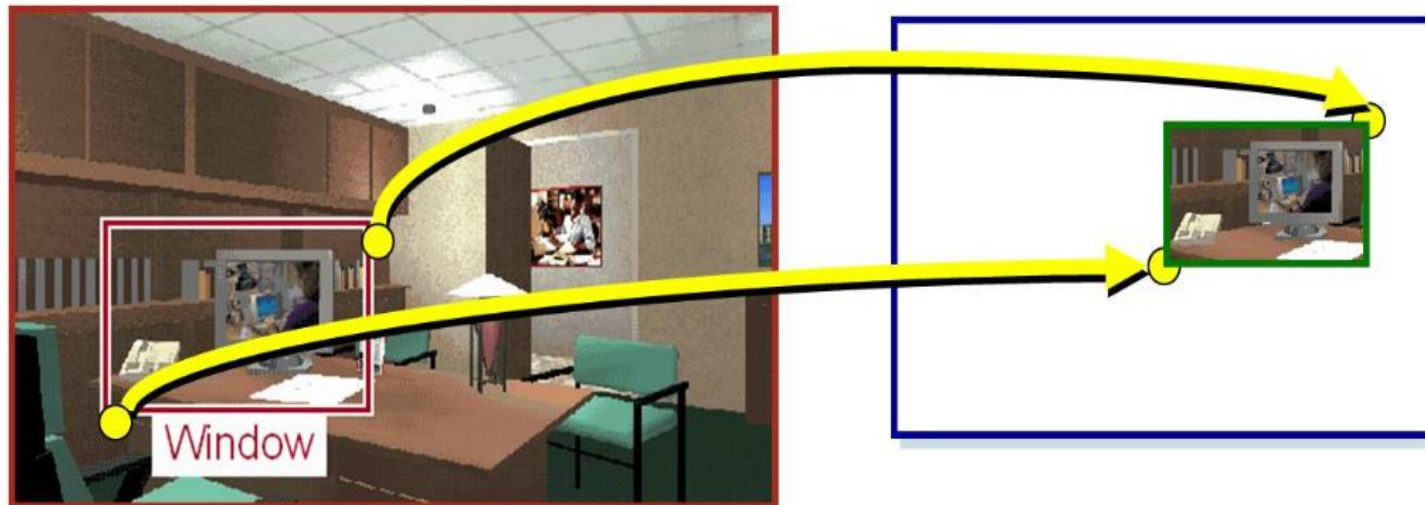
A View-Port is an area on a display device. Where we map a window. So, Viewport defines **where is to be viewed**. We also have to decide that at which portion of the display device, we want to view a particular scene.



- The **mapping** of a part of a **world-coordinate** scene to **device coordinates** is referred to as a **viewing transformation**.
- **2D viewing transformation** is simply referred to as the **window-to-viewport transformation** or the **windowing transformation**.

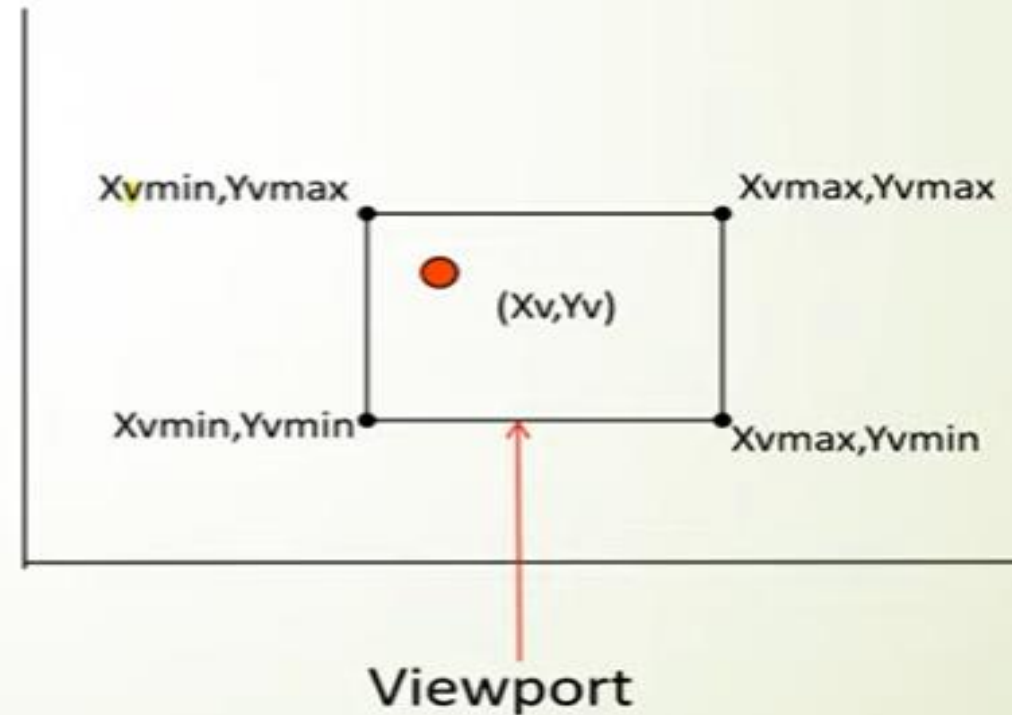
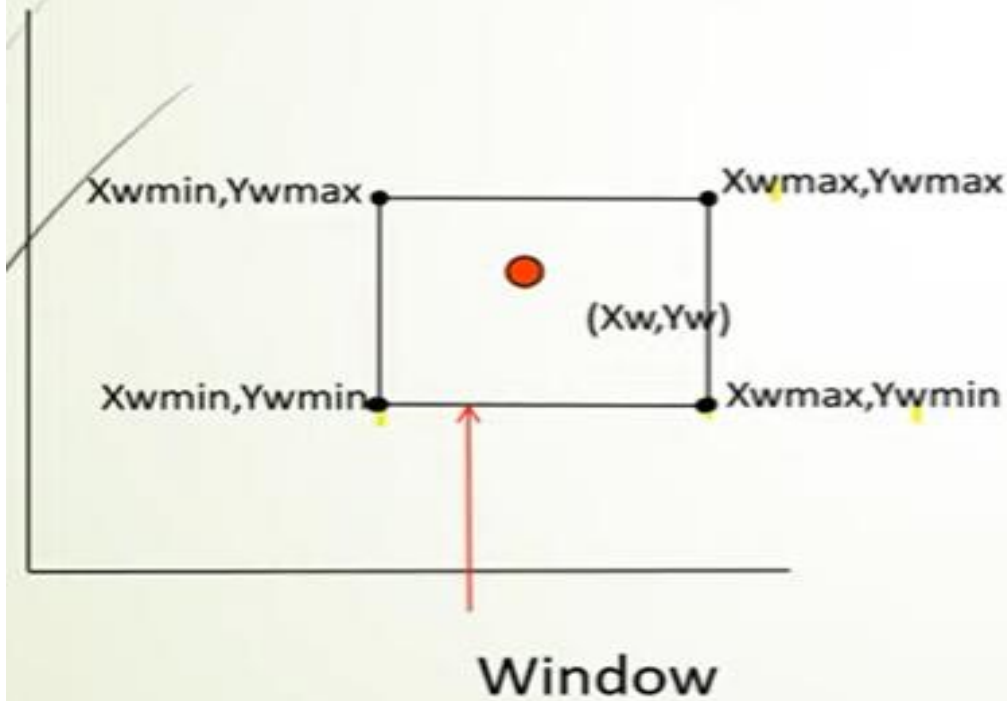
Window to View-Port Transformation

Now there are two coordinate systems. One is the World Coordinate system. While other is the Device Coordinate system. Therefore, The mapping of a world coordinate scene to device coordinate is **Window to Viewport Transformation** or **Windowing transformation**.



Window to view port transformation

For every point (X_w, Y_w) in the window there is a **corresponding or equivalent or relative point** (X_v, Y_v) in view port



View port transformation

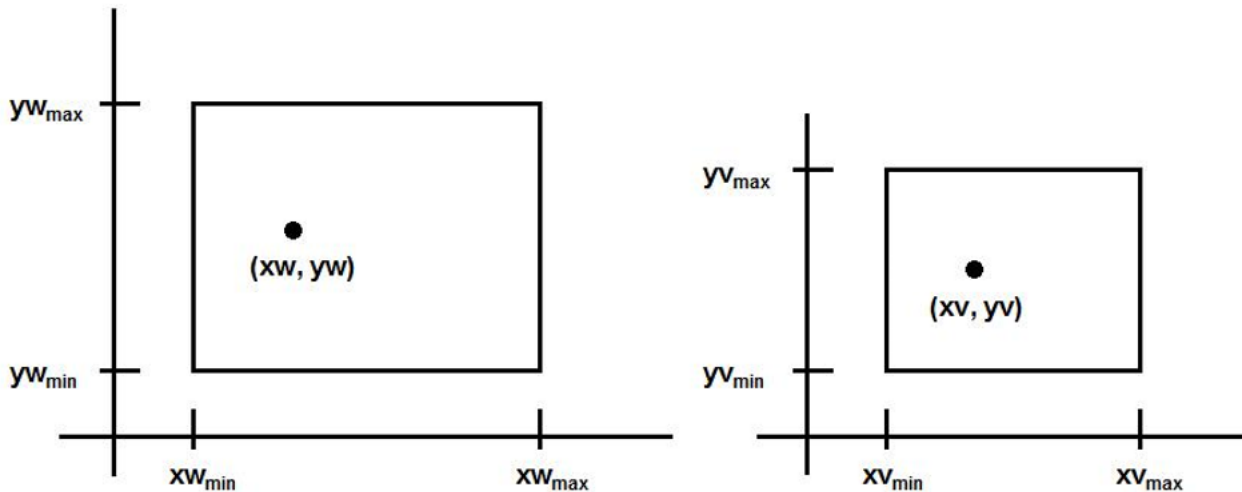
Mathematical Calculation of Window to Viewport:

It may be possible that the size of the Viewport is much smaller or greater than the Window. In these cases, we have to increase or decrease the size of the Window according to the Viewport and for this, we need some mathematical calculations.

(x_w, y_w) : A point on Window

(x_v, y_v) : Corresponding point on Viewport

- we have to calculate the point (x_v, y_v)



Normalized Point on Window $\left(\frac{x_w - x_{wmin}}{x_{wmax} - x_{wmin}}, \frac{y_w - y_{wmin}}{y_{wmax} - y_{wmin}} \right)$

Normalized Point on Viewport $\left(\frac{x_v - x_{vmin}}{x_{vmax} - x_{vmin}}, \frac{y_v - y_{vmin}}{y_{vmax} - y_{vmin}} \right)$

For x coordinate

$$\frac{X_w - X_{wmin}}{X_{wmax} - X_{wmin}} = \frac{X_v - X_{vmin}}{X_{vmax} - X_{vmin}}$$

For y coordinate

$$\frac{Y_w - Y_{wmin}}{Y_{wmax} - Y_{wmin}} = \frac{Y_v - Y_{vmin}}{Y_{vmax} - Y_{vmin}}$$

Solving these expressions for the viewport position (xv, yv), we have

$$X_v = X_{vmin} + (X_w - X_{wmin}) S_x$$

$$S_x = \frac{X_{vmax} - X_{vmin}}{X_{wmax} - X_{wmin}}$$

$$Y_v = Y_{vmin} + (Y_w - Y_{wmin}) S_y$$

$$S_y = \frac{Y_{vmax} - Y_{vmin}}{Y_{wmax} - Y_{wmin}}$$

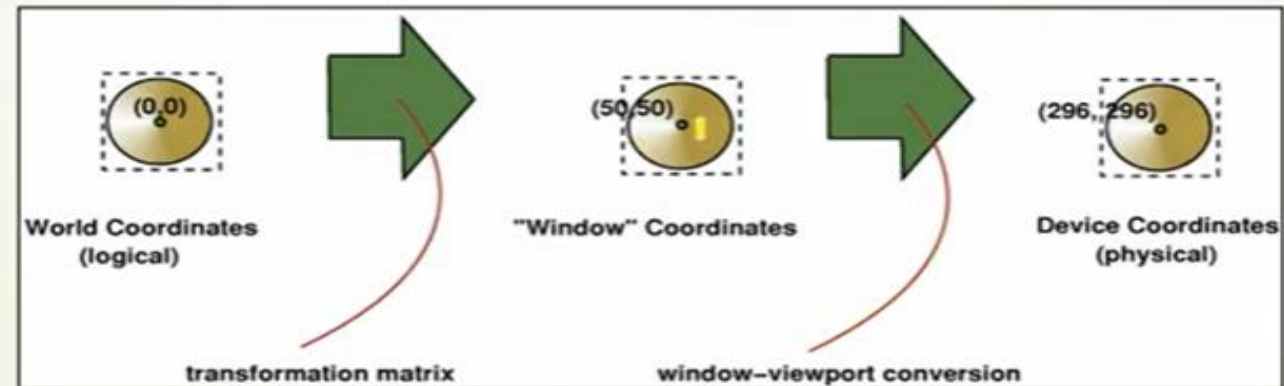
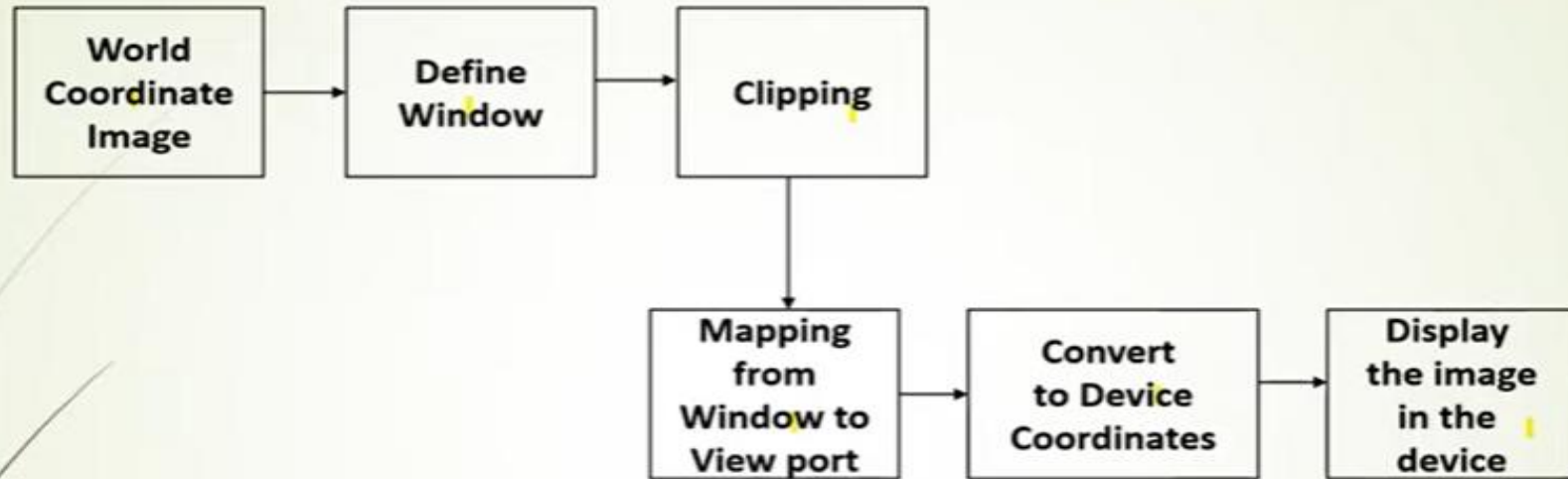
S_x- scaling factor for x coordinate

S_y- scaling factor for y coordinate

- **Window to Viewport Coordinate Transformation**

- **Equation 1** can also be derived with a set of transformations that converts the window area into the viewport area:
 1. Perform a **scaling** using a fixed-point position of (xw_{min}, yw_{min}) that scales the window area to the size of the viewport.
 2. **Translate** the scaled window area to the position of the viewport.
- Relative proportions of the objects are maintained if the scaling factors are the same ($s_x = s_y$).
- Otherwise world objects will be **stretched** or **contracted** in either the **x** or **y** direction when displayed on the output device.

Sequence of Viewing Transformation



Clipping

- It is an inside-outside procedure
 - Define the window area
 - Discarding portions outside window is called clipping



Clip Window



After Clipping

Before Clipping

Clipping Operations

Clipping Algorithm: Identifies those portions of a picture that are either inside or outside of a specified region of space.

Clip Window: The region against which an object is to be clipped.

- **Point** Clipping
- **Line** Clipping (straight-line segments)
- Area Clipping (**polygons**)
- **Text** Clipping

Classifications

- Point Clipping
- Line clipping
- Polygon clipping

Point clipping

Decide whether a point(x,y) is inside/outside a window

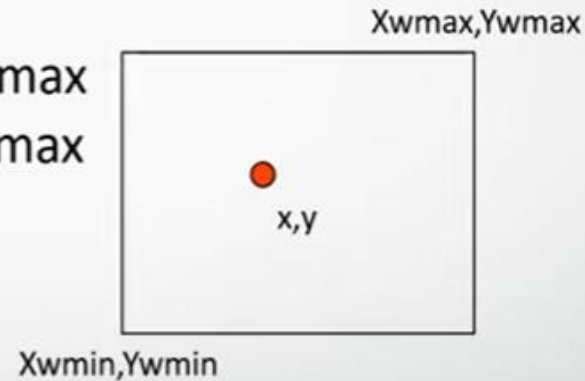
Conditions

x should fall between Xwmin & Xwmax

y should fall between Ywmin & Ywmax

$Xwmin \leq x \leq Xwmax$

$Ywmin \leq y \leq Ywmax$



means 4 **inequalities**

$x \geq Xwmin$

$x \leq Xwmax$

and

$y \geq Ywmin$

$y \leq Ywmax$

If point(x,y) satisfy all 4 inequalities
then

save the point

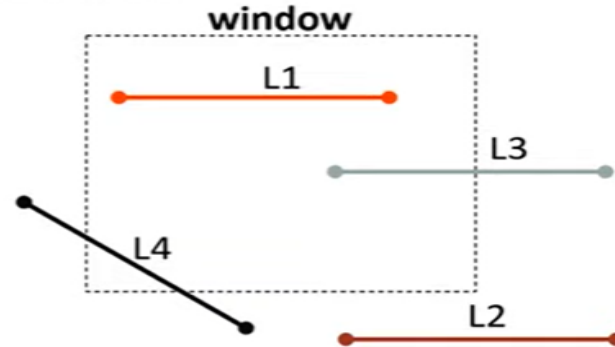
else

clip the point

BASICS

With respect to a window, the line can be

- * perfectly inside (L1)
- * perfectly outside (L2)
- * partial inside (L3)
- * endpoints outside, line segment inside (L4)

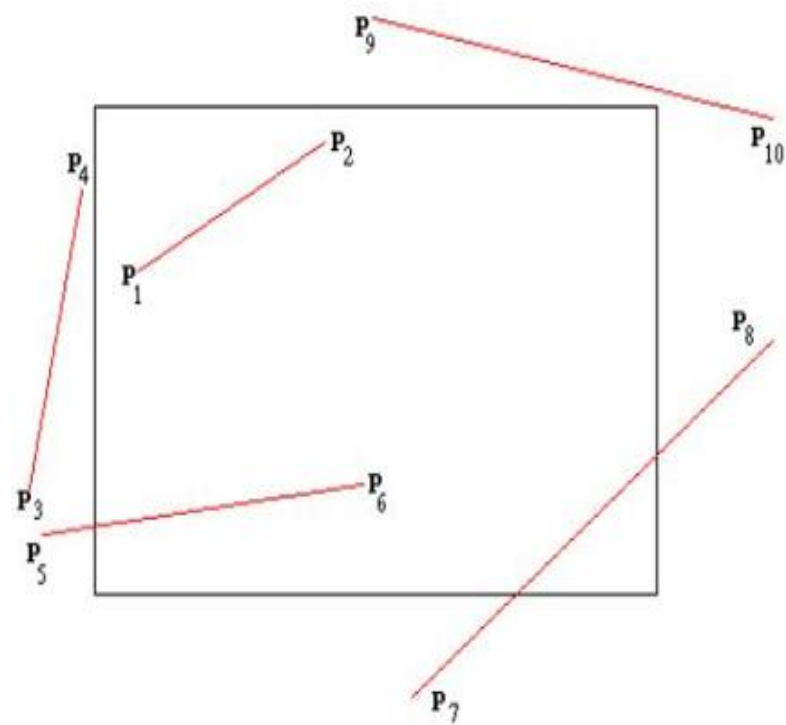


If the line is...

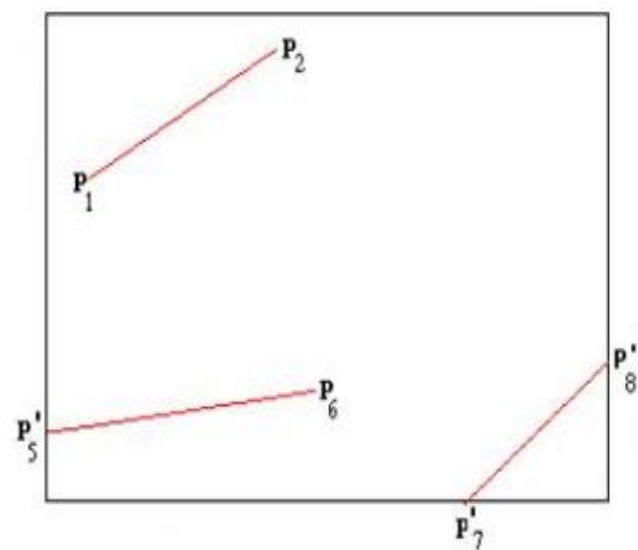
...perfectly inside (L1) --- **accept the line**

...perfectly outside (L2) --- **reject the line**

...crosses window boundary --- **clip the line**



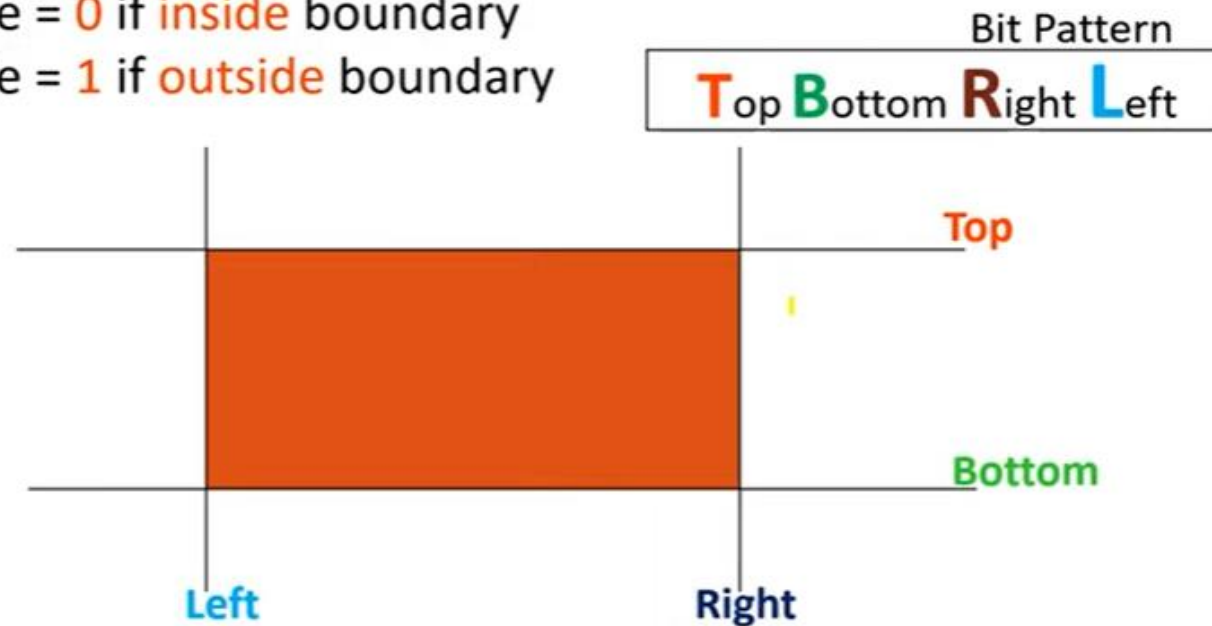
before clipping



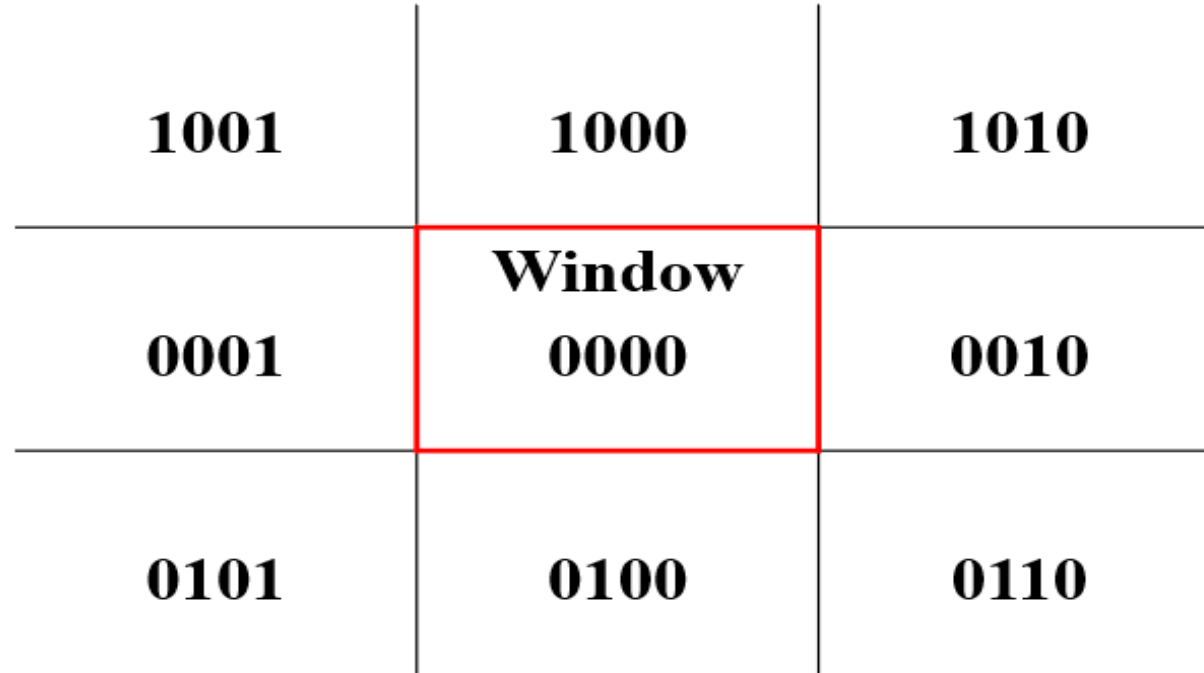
after clipping

Cohen-Sutherland Line Clipping Algorithm

- Uses a **coding scheme** where each endpoint is assigned a **region code**
- Will specify the endpoint w.r.t. to each window boundary
- Region code = 0 if **inside** boundary
- Region code = 1 if **outside** boundary



Cohen-Sutherland Line Clipping



Bit 4	Bit 3	Bit 2	Bit 1
Above	Below	Right	Left

Cohen-Sutherland Line Clipping Algorithm...

Encode the endpoints of the line

- If the 2 endpoints have the code 0000 & their AND is 0000 then line is completely inside. So **Accept the line**
- If the 2 endpoints are non zero & their AND is non zero ,the line is completely outside . So **Reject the line**
- If the 2 end points are non zero & their AND is zero ,the line is partially inside. So **clip the line**.

Clipping needs intersection point(s)

If a point is outside any window boundary **find the intersection point** on window boundary

Left $x = x_{\min}$

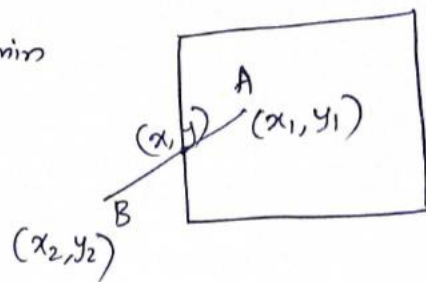
$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

$$= \frac{y - y_1}{x - x_1}$$

$$= \frac{y - y_1}{x_{\min} - x_1}$$

$$y - y_1 = m(x_{\min} - x_1)$$

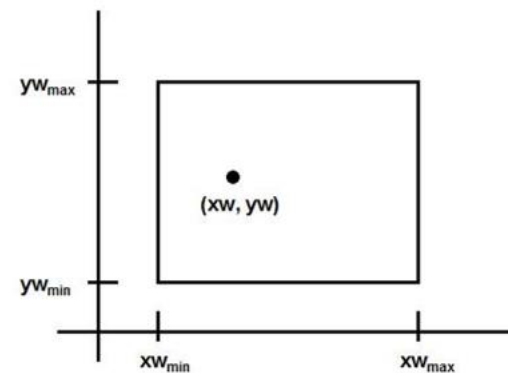
$$y = y_1 + m(x_{\min} - x_1)$$



Right

$x = x_{\max}$

$$y = y_1 + m(x_{\max} - x_1)$$



Top

$y = y_{\max}$

$$m = \frac{y - y_1}{x - x_1} = \frac{y_{\max} - y_1}{x - x_1}$$

$$(x - x_1) \cdot m = y_{\max} - y_1$$

$$x - x_1 = (y_{\max} - y_1) / m$$

$$x = x_1 + (y_{\max} - y_1) / m$$

Bottom

$y = y_{\min}$

$$x = x_1 + \frac{(y_{\min} - y_1)}{m}$$

*line L1

ENDPOINTS

0000

0000

-----AND

0000

Completely inside

*Line L2

ENDPOINTS

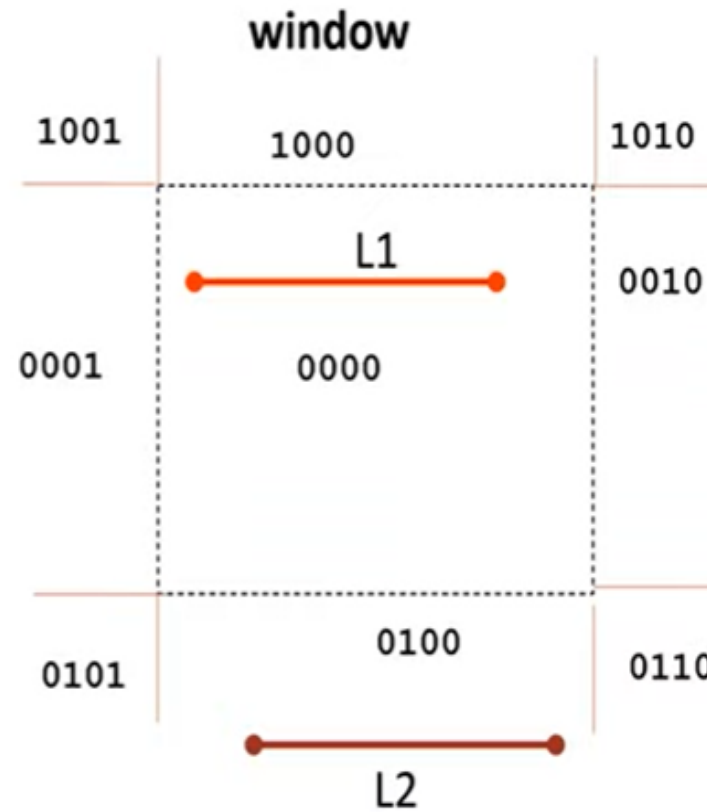
0100

0100

-----AND

0100

Completely Outside



*line L3

ENDPOINTS

0000

0010

-----AND

0000

Partially inside .need clipping

Find intersection point

New ENDPOINTS

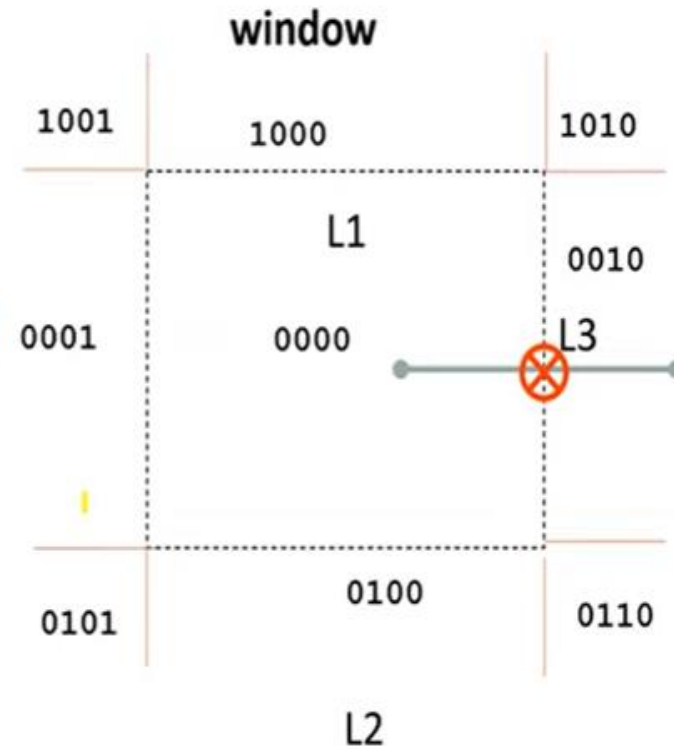
0000

0000

-----AND

0000

Completely Inside=>Accept



*line L4

ENDPOINTS

0001

0100

-----AND

0000

Partially inside .need clipping

Find intersection point

New ENDPOINTS

0001

0000

-----AND

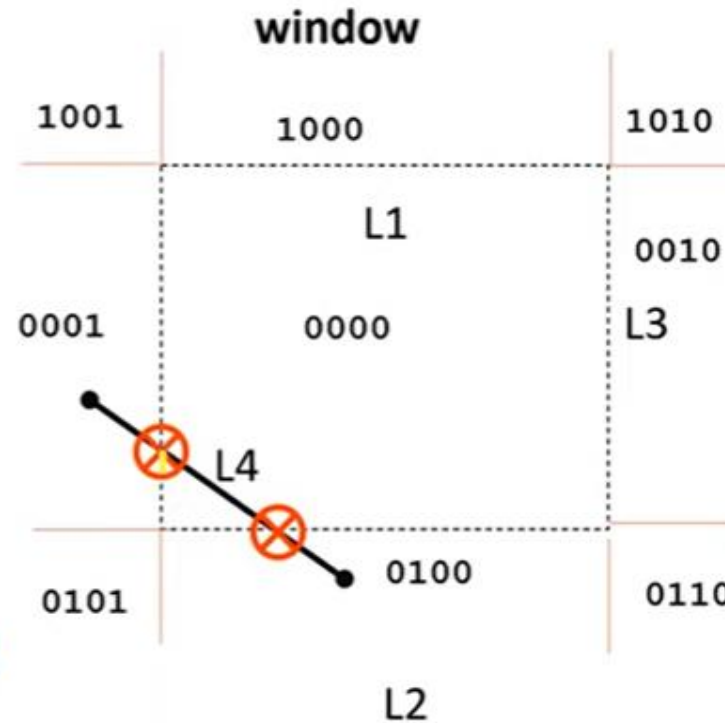
0000

Partially Inside. need clipping

Find intersection point

New endpoints

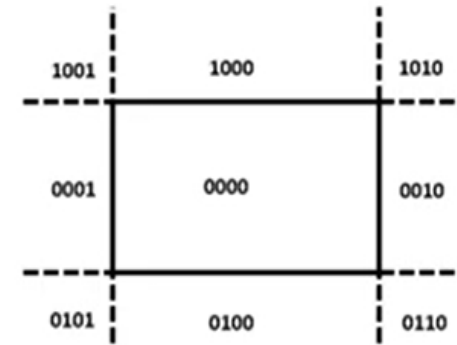
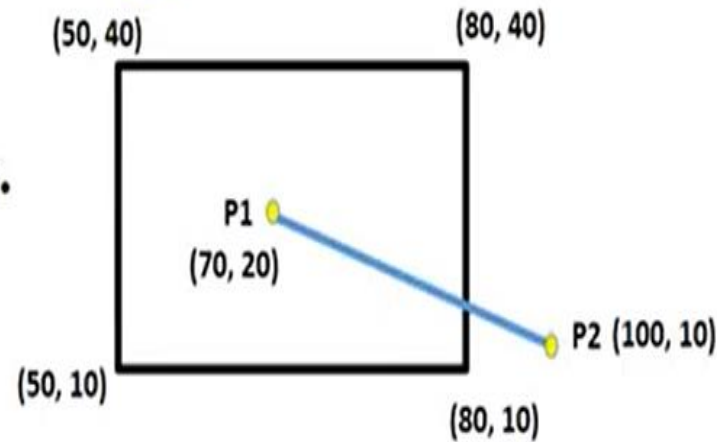
0000 & 0000 → 0000 → completely inside accept



Use the Cohen Sutherland Algorithm to Clip line P1(70,20), P2(100,10) against a window. Consider rectangular window ABCD, A(50,10), B(50,40), C(80,40) and D(80,10).

Step 1:

Draw window and Line.



$$P1 = 0000$$

$$P2 = 0010$$

$$\text{AND of P1 and P2} = 0000$$

Calculate Slope of Line (P1,P2).

$$\text{Slope of Line (P1,P2)} m = \frac{y_2 - y_1}{x_2 - x_1}$$

$$\text{Slope of Line (P1,P2)} m = \frac{10 - 20}{100 - 70} = -\frac{1}{3}$$

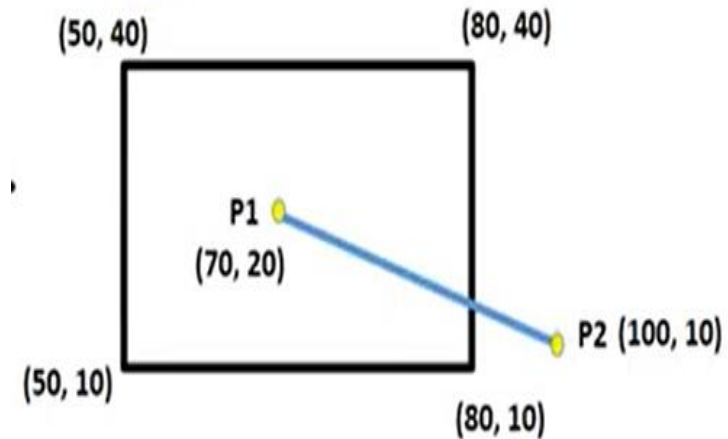
Line Intersection and Clipping

For **Left** window Edge, intersection point will be (x_L, y)

For **Right** window Edge, intersection point will be (x_R, y)

For **Top** window Edge, intersection point will be (x, y_T)

For **Bottom** window Edge, intersection point will be (x, y_B)



Find the Intersection of line P1,P2 with right edge of window i.e., Point P2.

Intersection point is found by solving the equation representing the line segment and boundary lines.

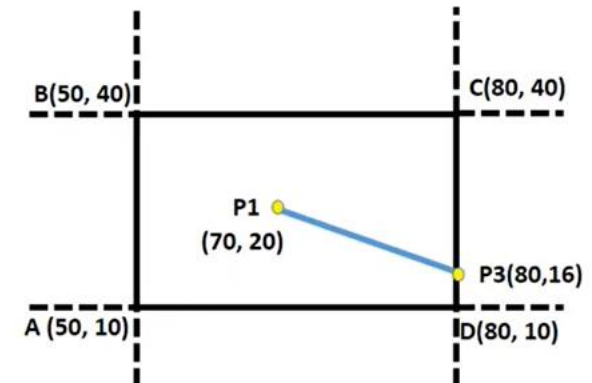
$x = 80$, Find y ?

Use point P2 $(x_2, y_2) = (100, 10)$

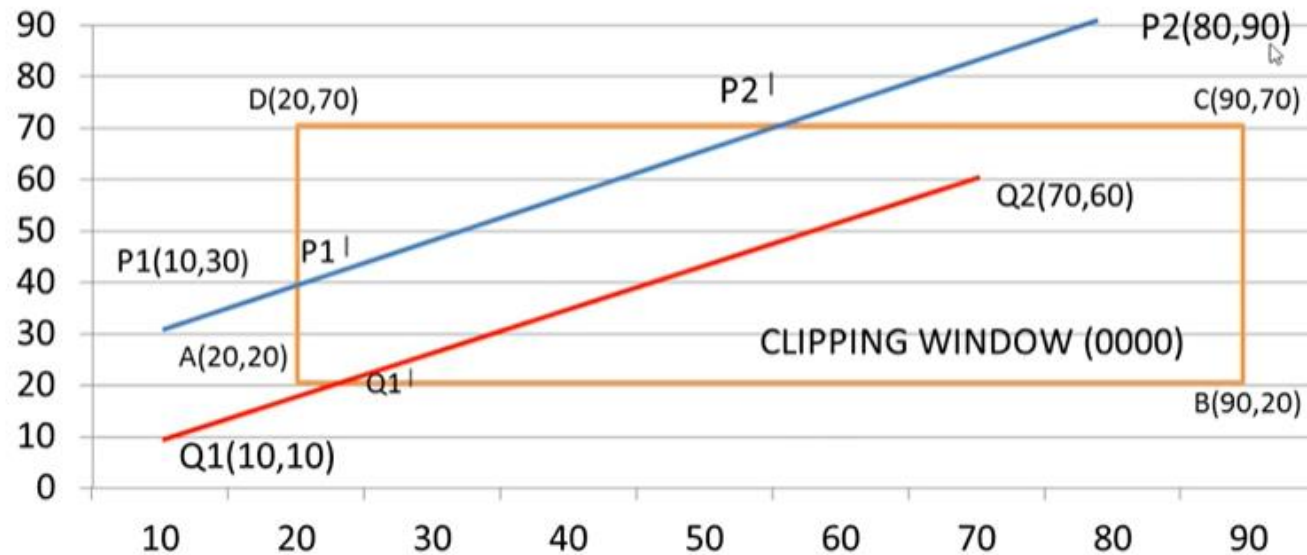
$$\text{Slope (m)} = \frac{y - y_2}{x - x_2}$$

$$-\frac{1}{3} = \frac{y - 10}{x - 100} = 16.66$$

Thus intersection point **P3 (80, 16.66)**.



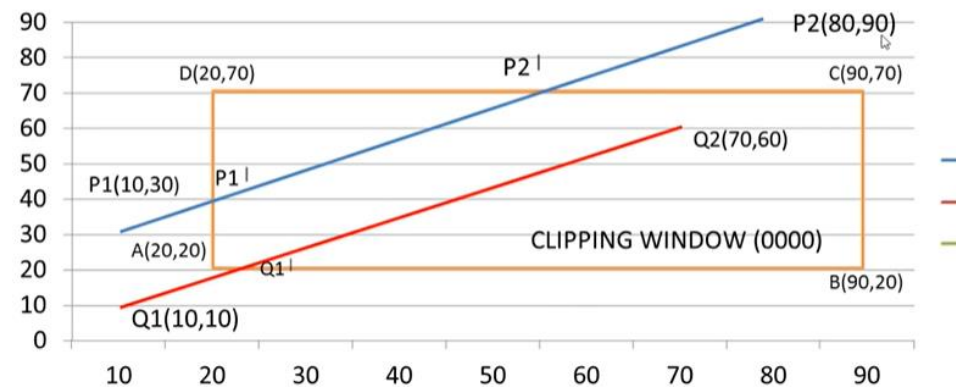
- Let ABCD be a rectangle window with A(20,20), B(90,20), C(90,70), D(20,70). Find the region codes for the end points & use Cohen Sutherland line Clipping algorithm to clip the following lines P1P2 with p1(10,30) & P2(80,90) & Q1Q2 with Q1(10,10) & Q2(70,60).



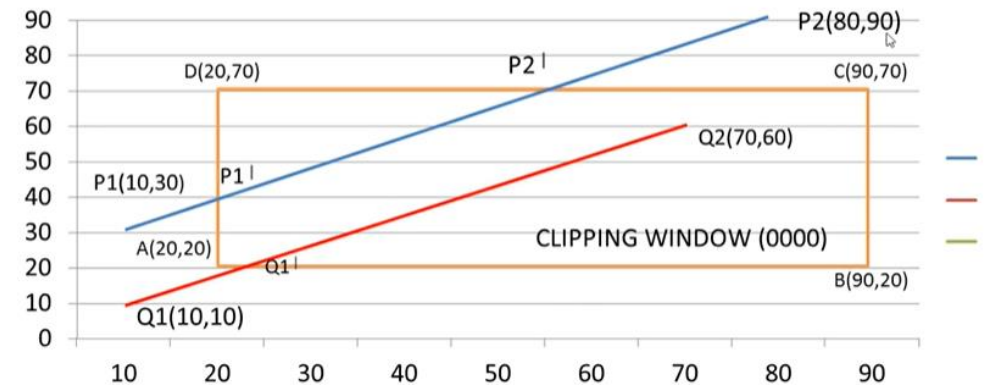
- Let's check the line P1P2
- P1-(10,30) P2-(80,90)
- As clipping window is 0000 and P1 lies to the left of the window the area code of P1 is 0001
- P2 lies at the top of the clipping window so its area code is 1000.
- Let's LOGICAL AND AREA CODES OF P1 & P2.
- $0001 \wedge 1000 = 0000$

- Since the logical AND operation of P1&P2 is 0000 P1P2 lies partially inside the window.
- Now lets find out location of P1[|] & P2[|] using slope of the line P1P2.
- $slope = \frac{y_2 - y_1}{x_2 - x_1}$
- Slope of P1P2 = $\frac{90 - 30}{80 - 10} = \frac{60}{70}$

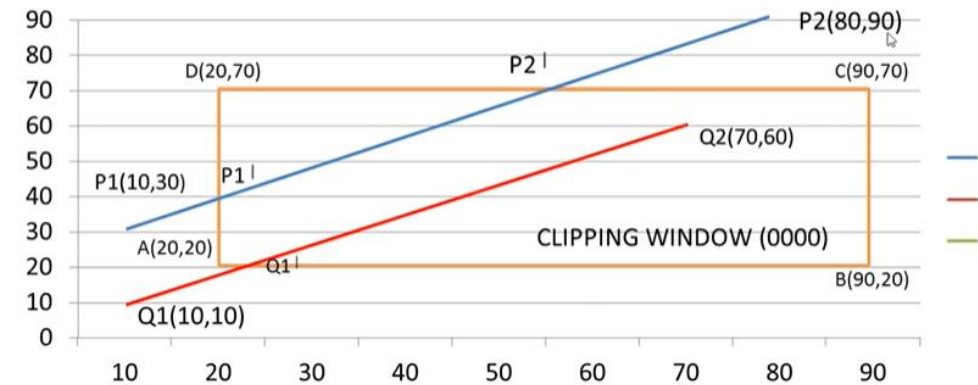
- Now we know the slope. Since P1[|] is a point on line P1P2, the slope will remain same for P1[|]P2.
- Since P1[|] is on line AD the x-co-ordinate of P1[|] is 20. Now we only need to find its y-co-ordinate.
- Lets again use the SLOPE formula but this time for P1[|]P2.
- Slope of P1[|]P2 = $\frac{90 - y_1}{80 - 20} = \frac{60}{70}$
- $90 - y_1 = 60 \times 60 / 70$
- $90 - y_1 = 3600 / 70$
- $y_1 = 38.57$
- For simplicity lets keep it 38.
- Therefore co-ordinates of P1[|] are (20,38).



- Similarly let's find co-ordinates of $P2^I$.
- Since $P2^I$ is on line DC, its y-co-ordinate will be 70.
- So let's use the slope formula again for line $P2^I P2$.
- Slope of $P2^I P2 = \frac{90-70}{80-x1} = \frac{60}{70}$
- $\frac{20}{80-x1} = \frac{60}{70}$
- $\frac{1400}{60} = 80 - x1$
- $x1 = 80 - 23.33 = 56.67 = 56$ approximately
- Therefore the co-ordinates of $P2^I$ are (56,70)



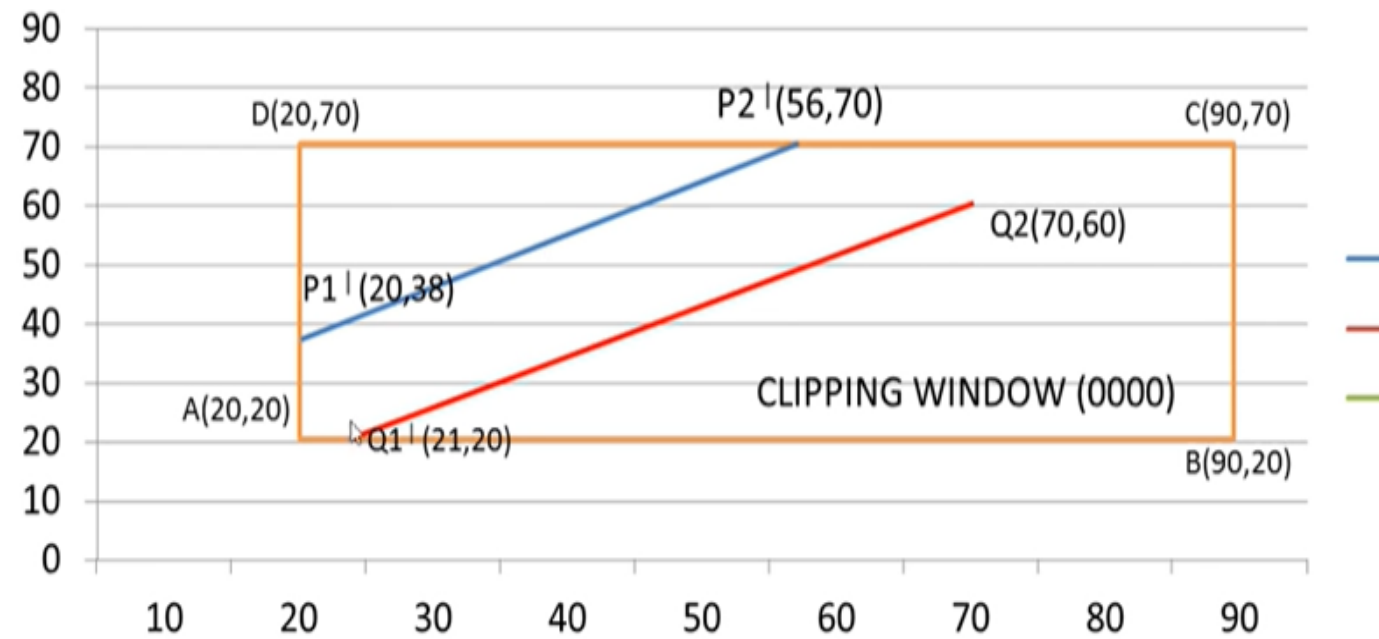
- Now let's take a look at line Q1Q2.
- Q1 lies outside the clipping window and its code will be 0101 as it's in the bottom left corner as compared to clipping window.
- The code of Q2 will be 0000 as it lies inside the clipping window.
- Let's perform AND operation on codes of Q1&Q2.
- $0101 \wedge 0000 = 0000$.
- So line Q1Q2 lies partially inside the window.



- Since Q1 lies outside the clipping window, it will be clipped and we will have only Q1'Q2 inside the window.
- To find co-ordinate positions of Q1' we first find the slope.
- $slope = \frac{y_2 - y_1}{x_2 - x_1}$
- Where x_1 & y_1 are co-ordinates of Q1 and x_2 & y_2 are co-ordinates of Q2.
- $slope = \frac{60 - 10}{70 - 10} = \frac{50}{60} = 0.83$

- Now as Q1' lies on line AB its y-coordinate will be 20, to find its x-coordinate let's use the slope formula.
- For Line Q1'Q2 slope will be same as Q1Q2 as it's part of the same line.
- $slope = \frac{60 - 20}{70 - x_1} = \frac{40}{70 - x_1} = 0.83$
- $70 - x_1 = \frac{40}{0.83}$
- $x_1 = 70 - 48.19 = 21.80$
- For simplicity let's assume it as 21.
- Therefore the co-ordinates of Q1' are (21,20)

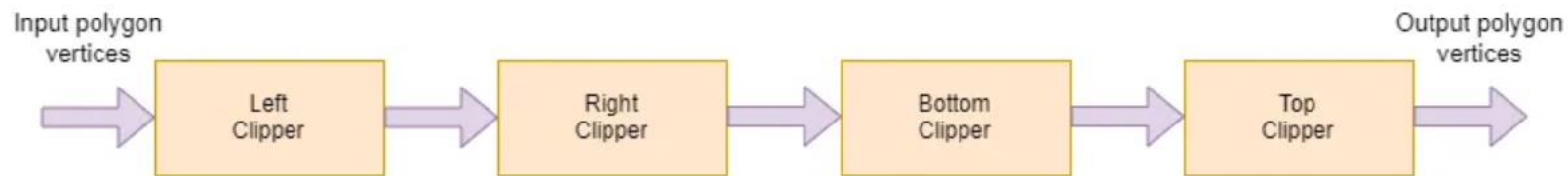
FINAL OUTCOME AFTER CLIPPING



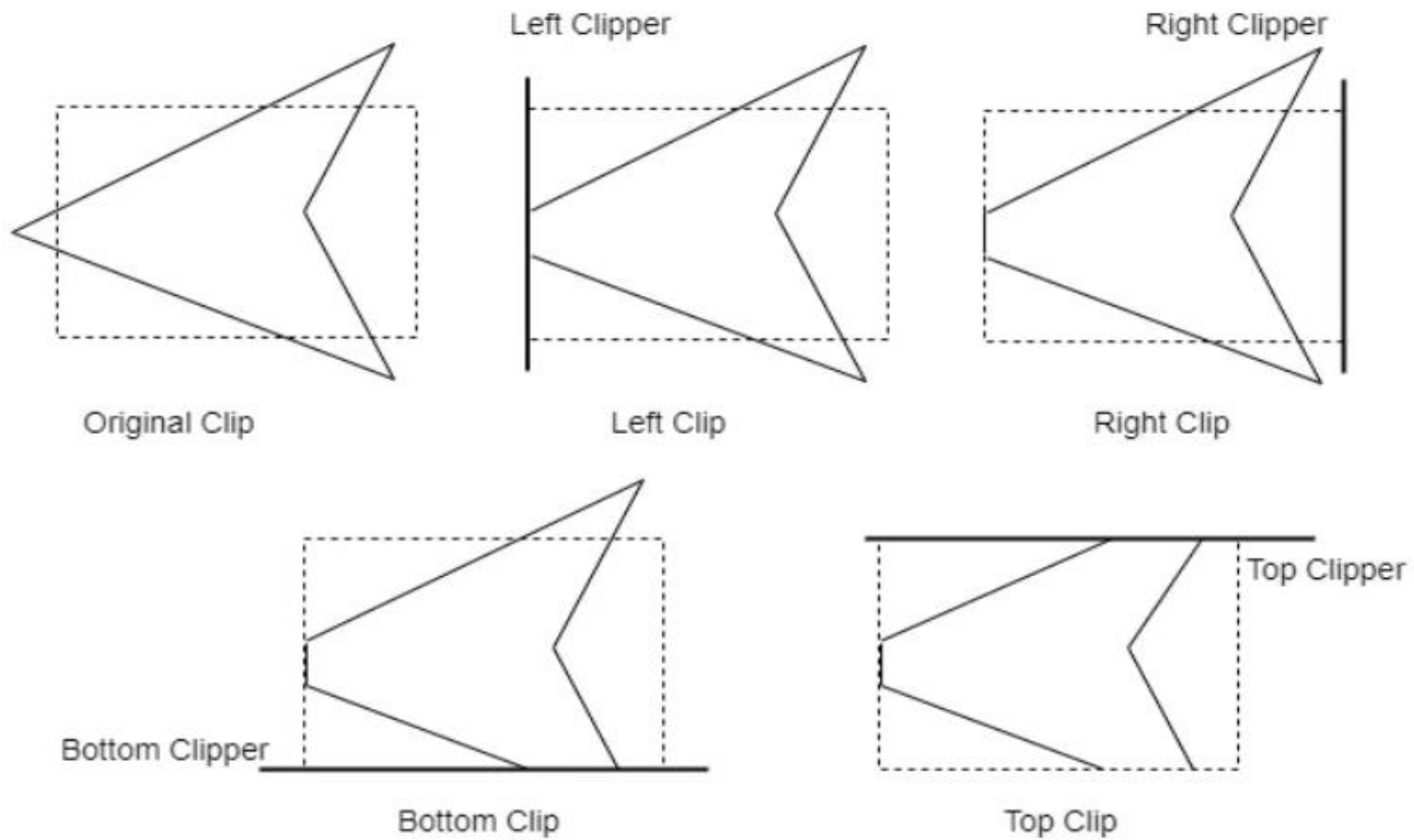
Sutherland-Hodgeman Polygon Clipping Algorithm

- Sutherland-Hodgeman Polygon Clipping Algorithm purposed by Sutherland and Hodgeman in 1974 to determine which portions of the polygon were visible within a window and clipping out the portions that are outside the window.
- This algorithm accepts as input a series of polygon vertices, $V_1, V_2, \dots, V_n$. These vertices define polygon edges from V_i to V_{i+1} and from V_n to V_1 .

- The algorithm clips a polygon by processing the polygon boundary as a whole against each window edge. It means all the vertices are processed for **clipping** against **each individual clip boundary** of clipping window one by one.
- It means starting with the initial set of vertices, the polygon is first clipped against left window boundary and a new set of vertices is produced.
- This set is given as input to next clipping boundary and the polygon is clipped against right clipping boundary to produce a new set of vertices and is then clipped against bottom and top clipping boundary to produce final set of vertices . ***[Left-Right-Bottom-Top]***



Thus, at each step a new set of output vertices is generated and is passed to the next window boundary clipper.

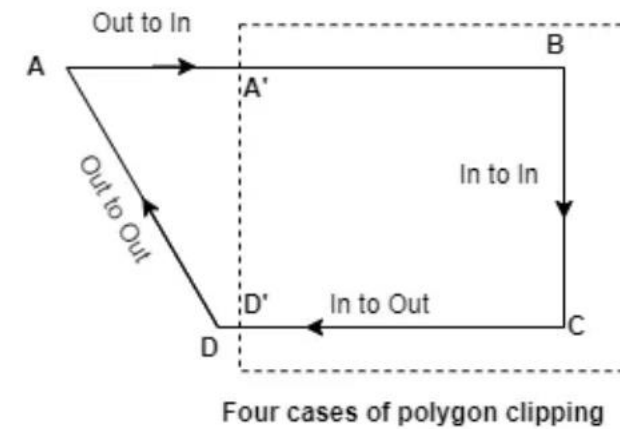


Clipping Polygon against all four clip boundaries

Four possibilities for a successive pair of two vertices of polygon

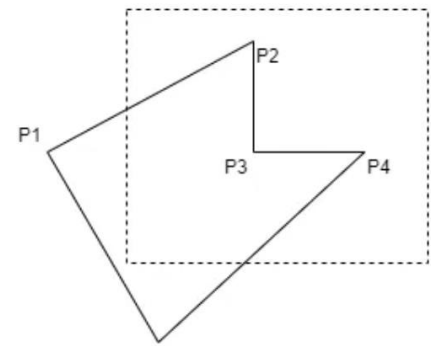
- **Case 1: out $\rightarrow$ in :**
 - If the starting vertex is outside the window boundary and ending vertex is inside then we preserve or save both point of intersection of polygon edge with window boundary and ending vertex in the output vertex list.
- **Case 2 : in $\rightarrow$ in :**
 - If both the starting and ending vertices are inside the window boundary then only the ending vertex is preserved or saved in output vertex list.
- **Case 3: in $\rightarrow$ out :**
 - If the starting vertex is inside the window and ending vertex is outside then only the point of intersection of polygon edge with window boundary is saved in output list.
- **Case 4: out $\rightarrow$ out :**
 - If both the starting and ending vertices are outside the window boundary then nothing is added or saved in the output vertex list.

Case	Output
Out → In	A' B [2 Vertices, Point of intersect with window and ending edge]
In → In	C [1 ending vertex]
In → Out	D' [Point of intersect with window]
Out → Out	N/A [Nothing is included]



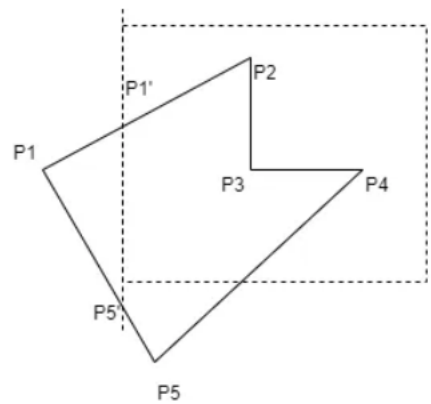
Example of Sutherland-Hodgeman Polygon Clipping Algorithm

Let us consider a polygon with vertices P1, P2, P3, P4, P5. Inside in a viewing window. Applying Sutherland-Hodgeman Polygon Clipping Algorithm on this polygon, we get:



Clipping polygon engaged left window boundary:

Here the input vertex list is P1, P2, P3, P4 P5 and the edges are P1P2, P2P3, P3P4, P4P5 and P5P1. Now, consider all the edge with respect to (w.r.t.) left window boundary.

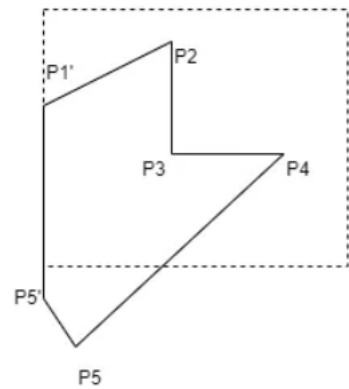


Using Left Clipper

Edge	Type	Output
P1P2	Out $\rightarrow$ In	P1'P2
P2P3	In $\rightarrow$ In	P3
P3P4	In $\rightarrow$ In	P4
P4P5	In $\rightarrow$ In	P5
P5P1	In $\rightarrow$ Out	P5'

Clipping polygon engaged right window boundary:

Here the input vertex list is P1', P2, P3, P4, P5, P5' and the edges are P1'P2, P2P3, P3P4, P4P5, P5P5' and P5'P1'. Now, consider all the edge with respect to (w.r.t.) right window boundary. There will be **no changes** because all edges lies inside the right clipping window.

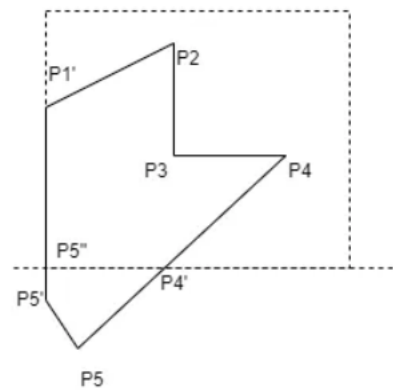


Using Right Clipper

Edge	Type	Output
P1'P2	In → In	'P2
P2P3	In → In	P3
P3P4	In → In	P4
P4P5	In → In	P5
P5P5'	In → In	P5'
P5'P1'	In → In	P1'

Clipping polygon engaged bottom window boundary:

Here the input vertex list is P1', P2, P3, P4, P5, P5' and the edges are P1'P2, P2P3, P3P4, P4P5 and P5P5'. Now, consider all the edge with respect to (w.r.t.) bottom window boundary.

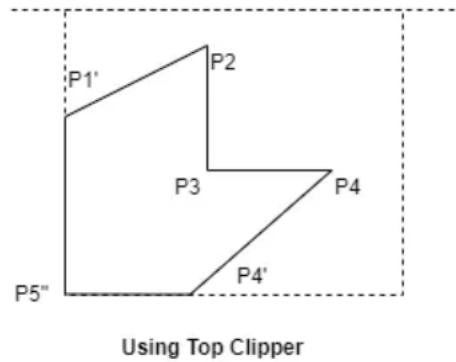


Using Bottom Clipper

Edge	Type	Output
P1'P2	In → In	P2
P2P3	In → In	P3
P3P4	In → In	P4
P4P5	In → Out	P4'
P5P5'	Out → Out	N/A
P5'P1'	Out → In	P5''

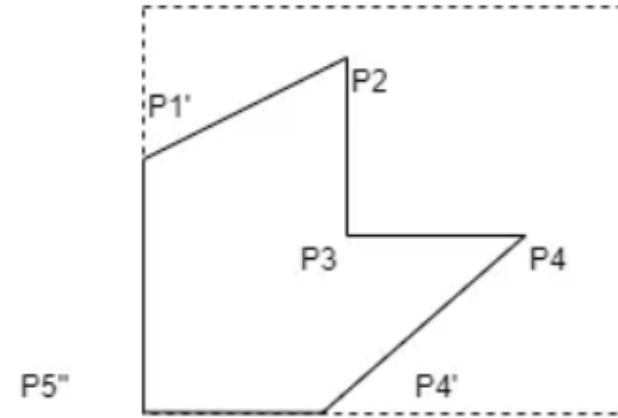
Clipping polygon engaged top window boundary:

Here the input vertex list is P1', P2, P3, P4, P4', P5'' and the edges are P1'P2, P2P3, P3P4, P4P4', P4'P5'' and P5''P1'. Now, consider all the edge with respect to (w.r.t.) top window boundary. Here the input vertex list is P1', P2, P3, P4, P5, P5' and the edges are P1'P2, P2P3, P3P4, P4P5, P5P5' and P5'P1'. Now, consider all the edge with respect to (w.r.t.) right window boundary. There will be **no changes** because all edges lies inside the top clipping window.

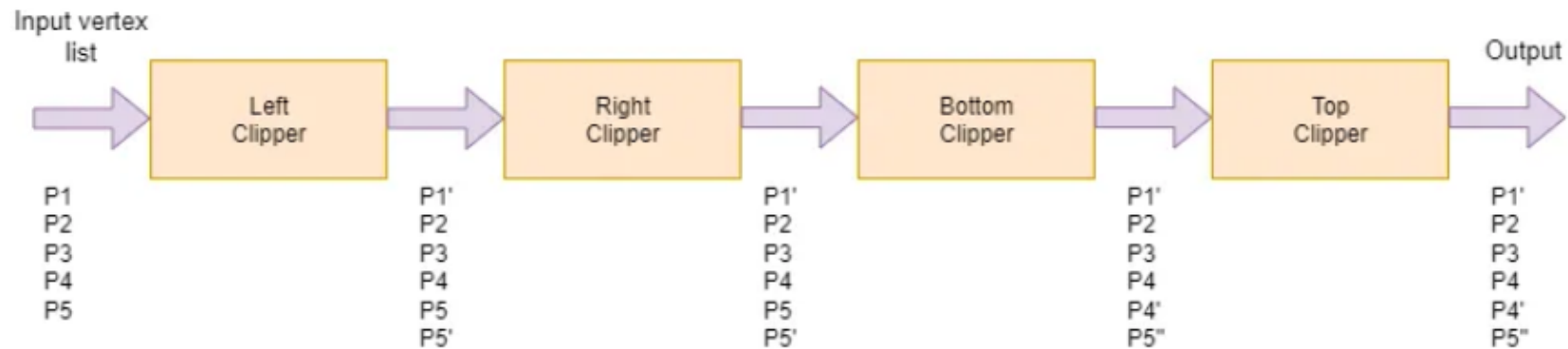


Edge	Type	Output
P1'P2	In → In	P2
P2P3	In → In	P3
P3P4	In → In	P4
P4P4'	In → In	P4'
P4'P5''	In → In	P5''
P5''P1'	In → In	P1'

Final output using Sutherland-Hodgeman Polygon Clipping Algorithm will be:



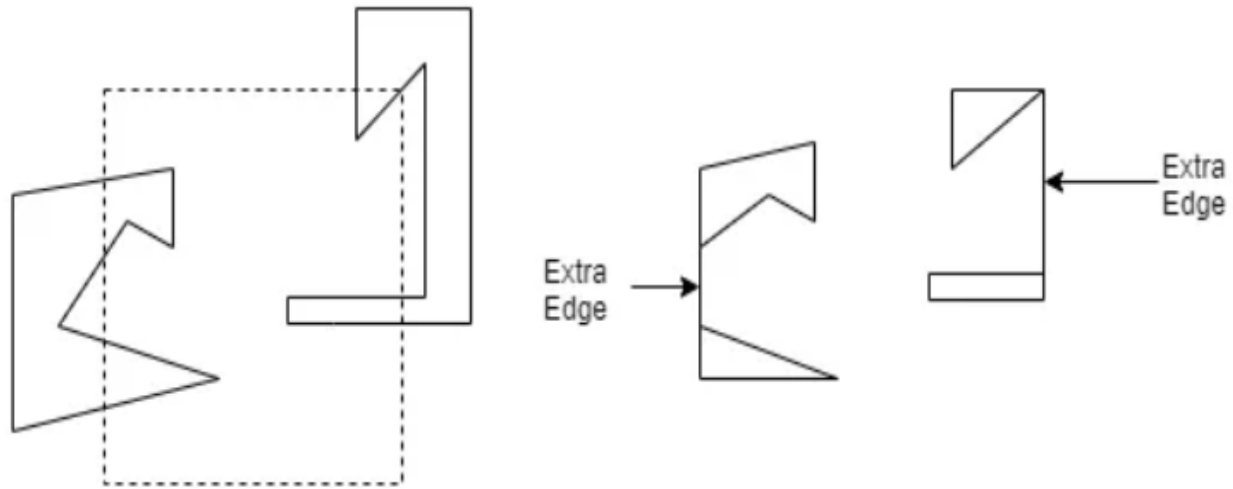
Final Output



Vertex list produced by various clippers

Limitations of Sutherland-Hodgeman Polygon Clipping Algorithm

1. In some concave polygons, the theorem creates extra edges while clipping.



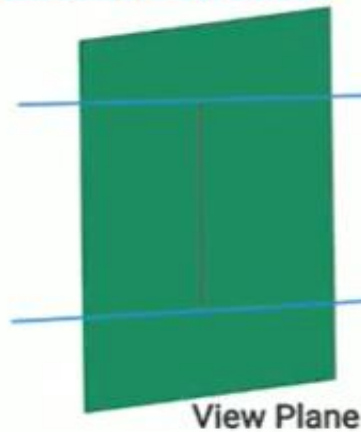
Concave polygon not clipped correctly

PROJECTION

It is the process of converting a 3D object into a 2D object. Or Mapping or transformation of the object in projection plane or view plane.

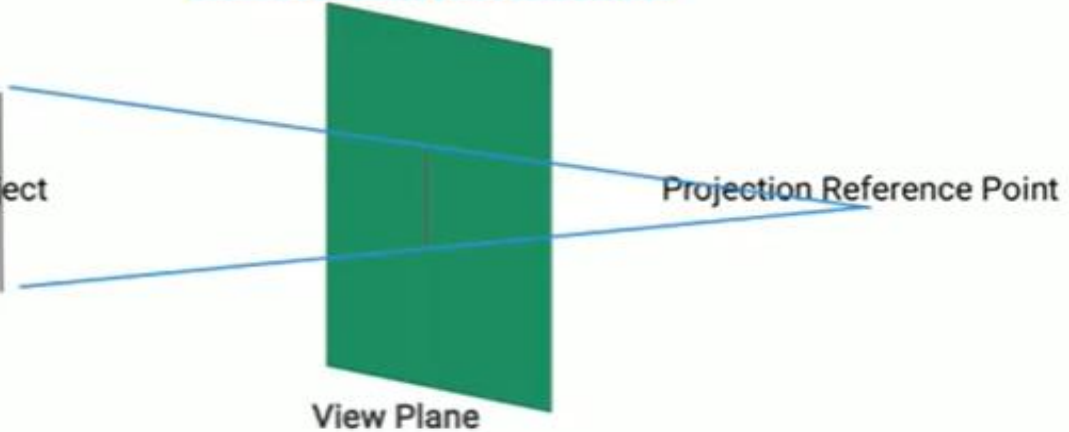
Two Types of Projection

1. Parallel Projection



It give Accurate not Realistic View.

2. Perspective Projection



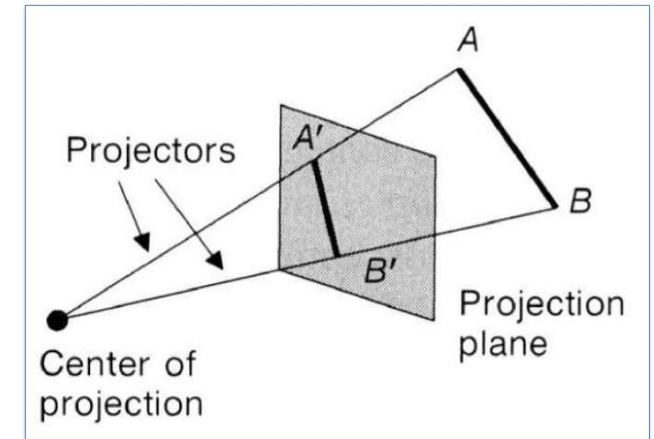
It give Realistic not Accurate View.

2 types of projections

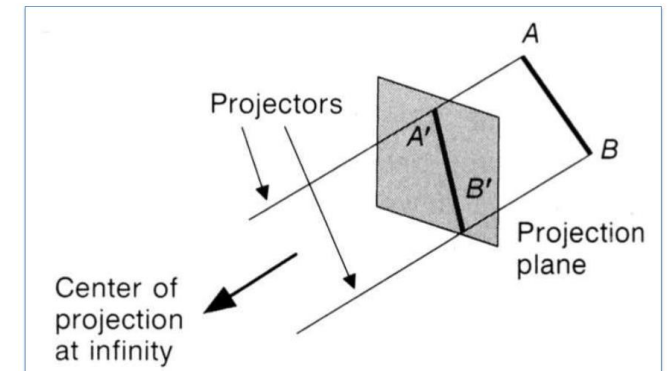
Perspective
Parallel

In **parallel projection**, coordinate positions are transformed to the view plane along parallel lines. In **perspective projection**, object position are transformed to the view plane along lines that converge to a point called **projection reference point (center of projection)**

Perspective Projection



Parallel Projection

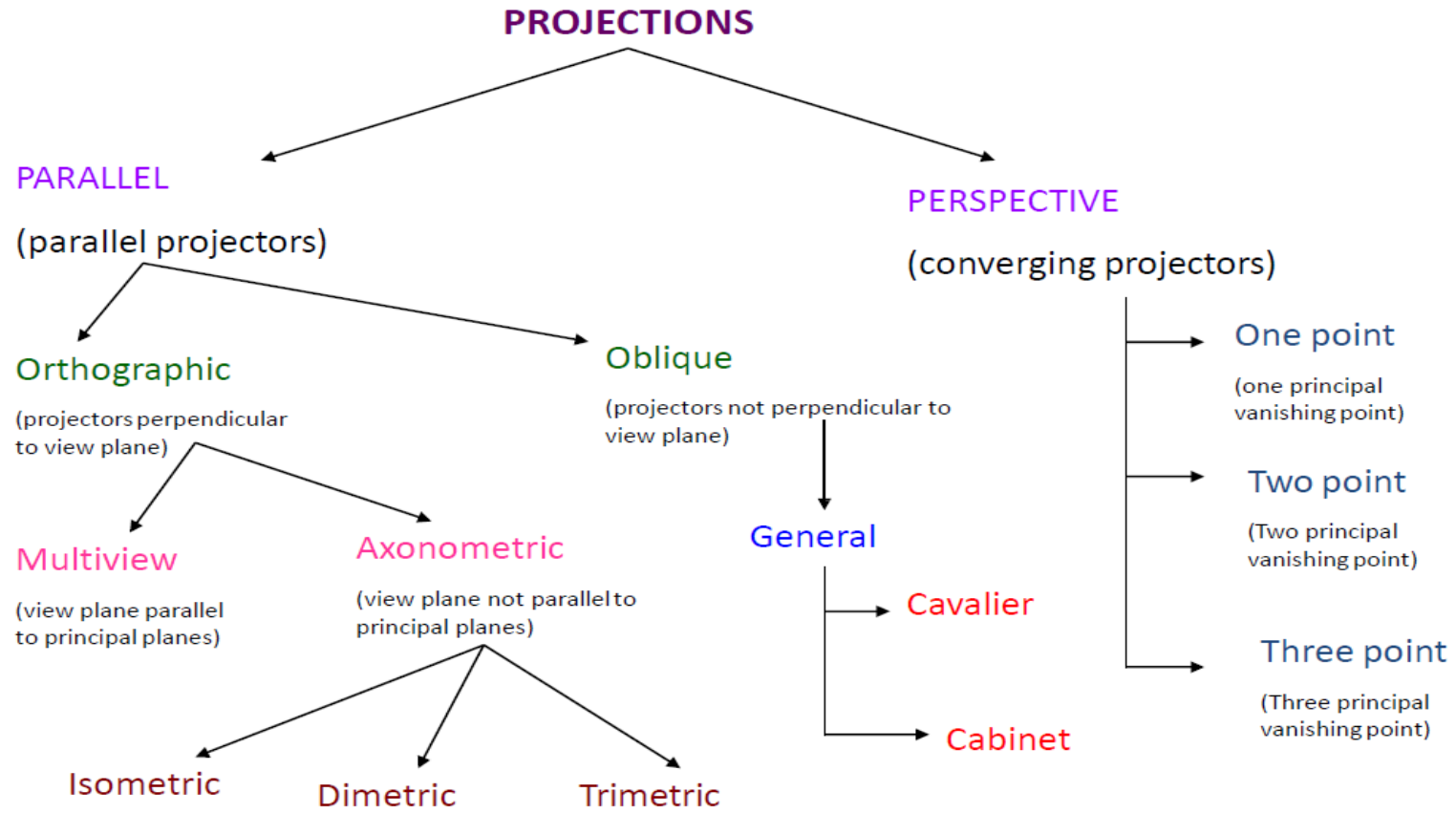


Perspective v Parallel

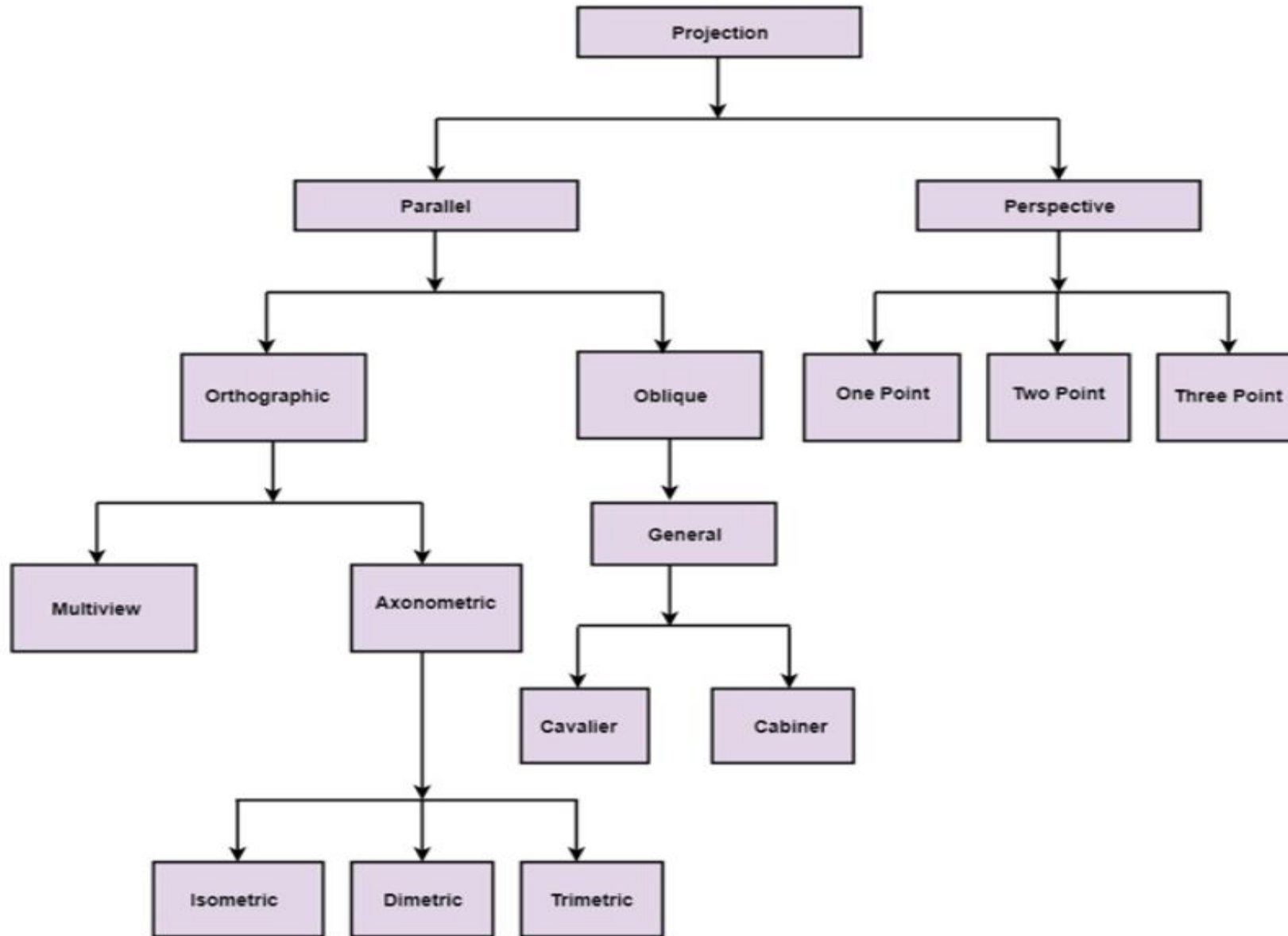
- **Perspective:**
 - visual effect is similar to human visual system...
 - has 'perspective foreshortening'
 - size of object varies inversely with distance from the center of projection. Projection of a distant object are smaller than the projection of objects of the same size that are closer to the projection plane.
- **Parallel:**
 - It preserves relative proportion of object.
 - less realistic view because of no foreshortening
 - however, parallel lines remain parallel.

BASIS OF DIFFERENCE	PERSPECTIVE PROJECTION	PARALLEL PROJECTION
Description	A perspective projection can be described as the projector lines (lines of sight) that converge at the center of projection, which results in many visual effects of an object.	A parallel projection is a projection of an object in three-dimensional space onto a fixed plane referred as the projection plane or image plane, where the rays, known as lines of sight or projection lines are parallel to each other.
Types	One-point perspective Projection. Two-point perspective projection. Three-point perspective projection.	Orthographic parallel projection Oblique parallel projection.
Accurate View Of Object	Perspective projection cannot give the accurate view of object.	Parallel projection can give the accurate view of object.

	Perspective	Parallel
Realistic View of Object	Perspective projection forms a realistic picture of object.	Parallel projection does not form realistic view of object.
Distance Of The Object From The Center Of Projection	The distance of the object from the center of projection is finite.	In parallel projection, the distance of the object from the center of projection is infinite.
Projector	Projector in perspective projection is not parallel.	Projector in parallel projection is parallel.
Preservation Of Relative Portion Of An Object	Perspective projection cannot preserve the relative proportion of an object.	Parallel projection can preserve the relative proportion of an object.
Lines Of Projection	The lines of perspective projection are not parallel.	The lines of parallel projection are parallel.



Types of Projection

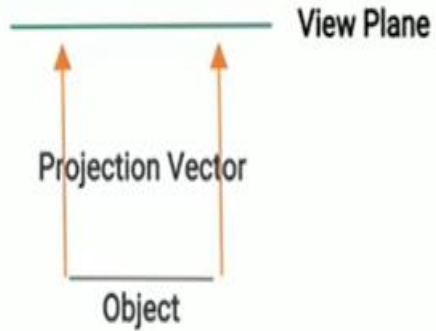


PARALLEL PROJECTION

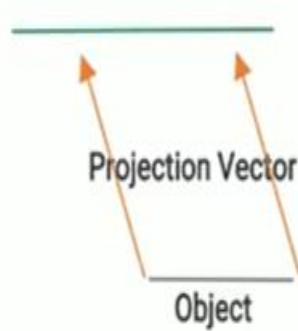
Parallel projection is used for displaying images in its true size and shape, these projections are made by coordinate positions are transformed to the view plane along parallel lines.

Two Types of Parallel Projections

1. Orthographic Projection



2. Oblique Projection

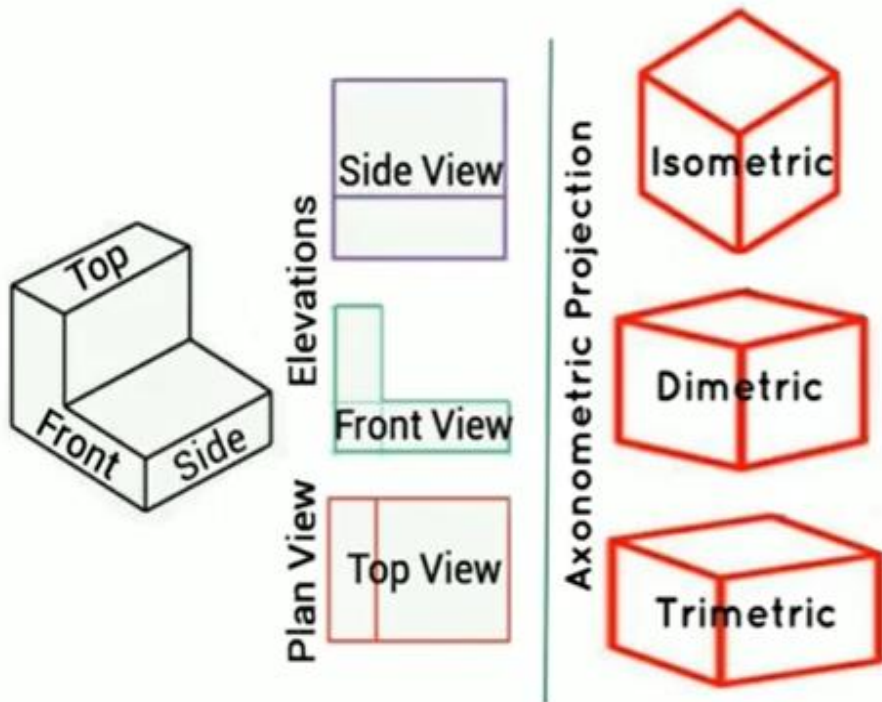


2 types:

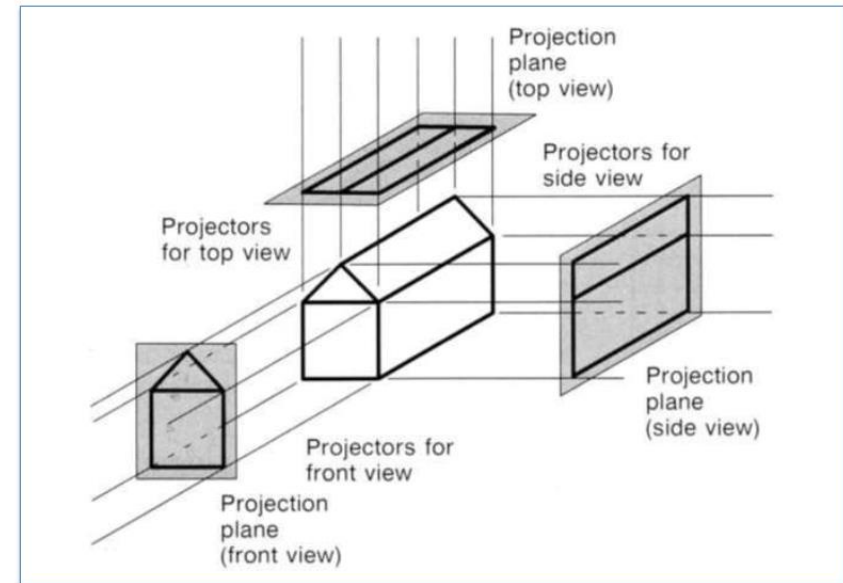
- **Orthographic** : when the projection is perpendicular to the view plane. In short,
 - direction of projection = normal to the projection plane.
 - the projection is perpendicular to the view plane.
- **Oblique** : when the projection is not perpendicular to the view plane. In short,
 - direction of projection $\neq$ normal to the projection plane.
 - Not perpendicular.

ORTHOGRAPHIC PROJECTION

- Used to produce Front, Side and Top views of an object.
- To display more than one face of an object called Axonometric orthographic projection.
Eg: Isometric projection, Dimetric projection, Trimetric projection



Orthogonal projections:

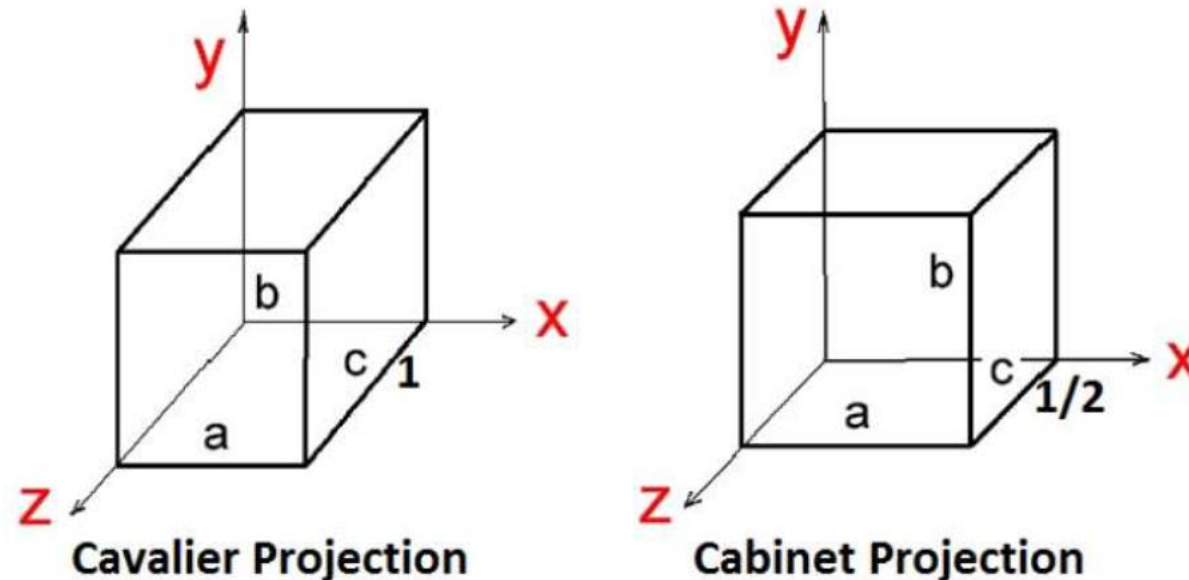


Oblique Projection

In oblique projection, the direction of projection is not normal to the projection of plane. In oblique projection, we can view the object better than orthographic projection.

There are two types of oblique projections – **Cavalier** and **Cabinet**. The Cavalier projection makes 45° angle with the projection plane. The projection of a line perpendicular to the view plane has the same length as the line itself in Cavalier projection. In a cavalier projection, the foreshortening factors for all three principal directions are equal.

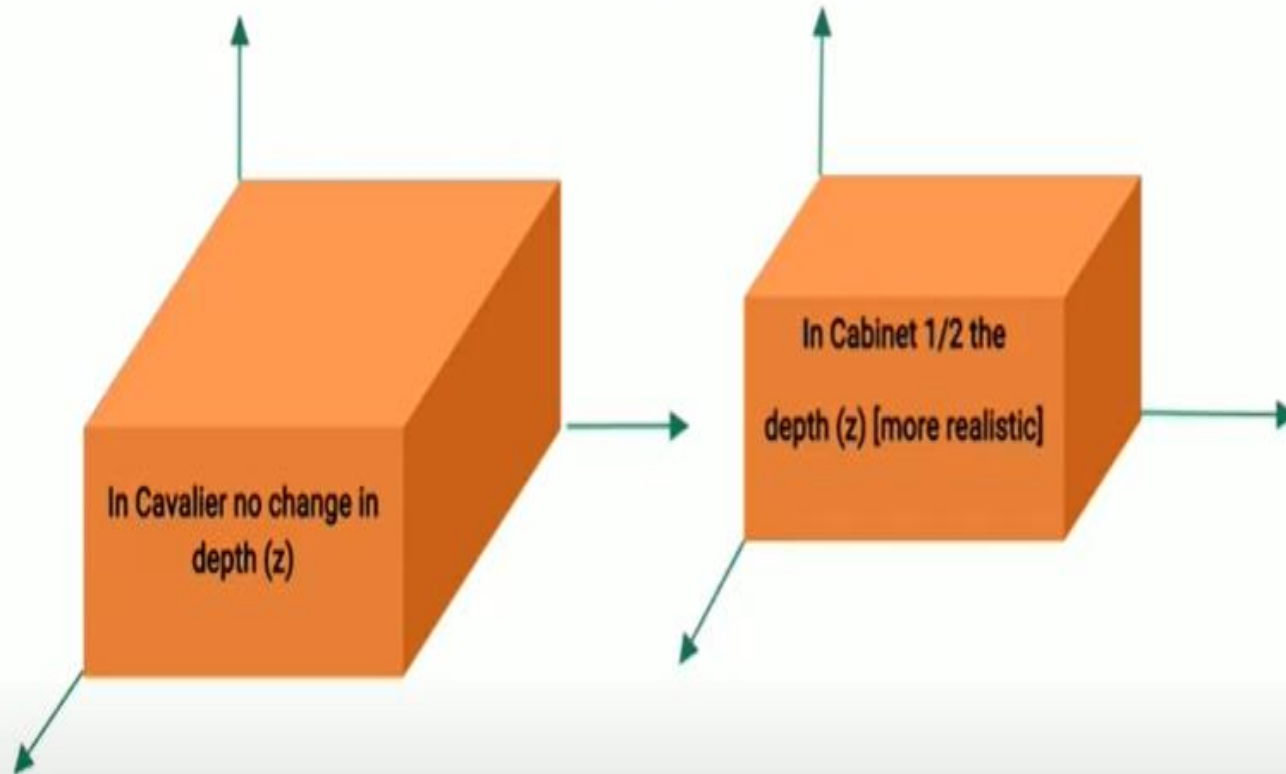
The Cabinet projection makes 63.4° angle with the projection plane. In Cabinet projection, lines perpendicular to the viewing surface are projected at $\frac{1}{2}$ their actual length. Both the projections are shown in the following figure –



OBLIQUE PROJECTION

Two common type of Oblique Projection are Cavalier and Cabinet

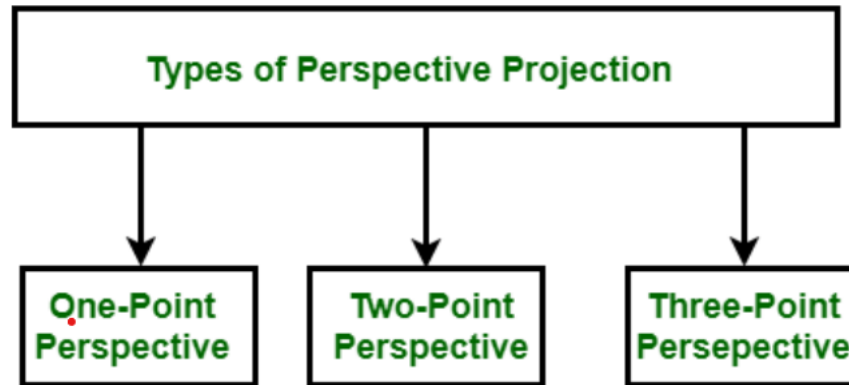
The difference between the two is how the depth (z) of the object is represented.



Cavalier Oblique Projection	Cabinet Oblique Projection
Angle between the plane of projection and the line of projection is 45° .	Angle between the plane of projection and the line of projection is $63^{\circ}26'$.
Length of projection of a line will have the same length as that of actual length	Length of projection of a line will have a length equal to half the actual length
Even though all the sides of cube are equal in length, the view has a distorted appearance	This view has less distorted appearance

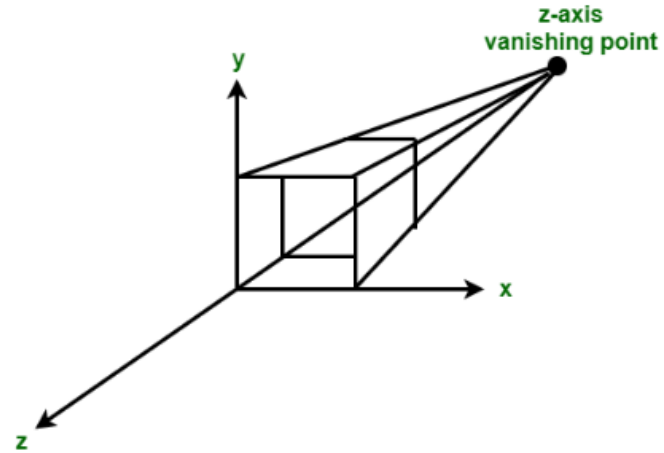
Types of perspective projections

- Types of perspective projections



- **One Point Perspective Projection –**

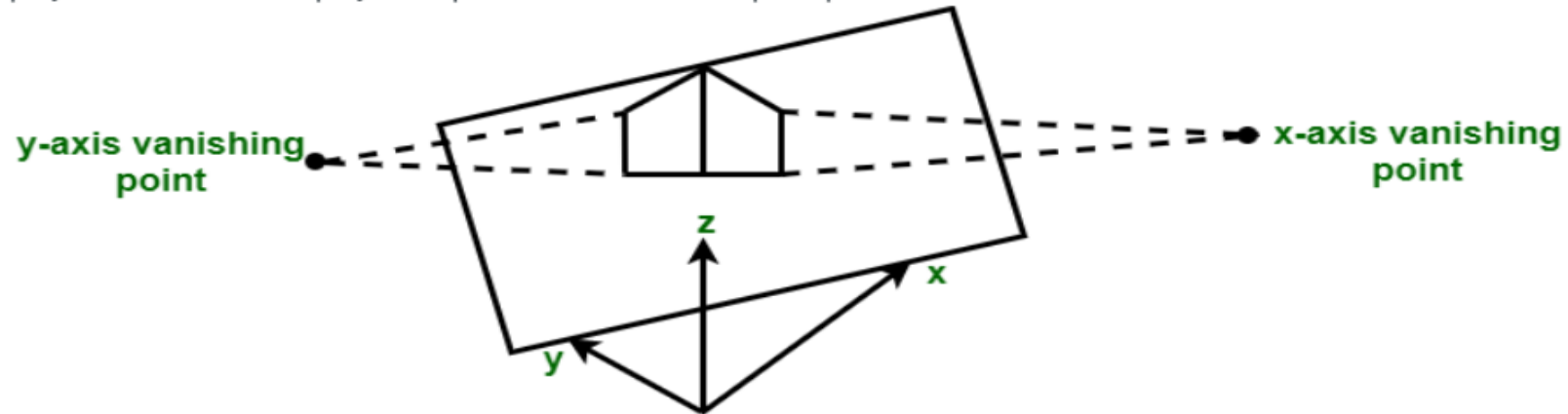
One point perspective projection occurs when any of principal axes intersects with projection plane or we can say when projection plane is perpendicular to principal axis.



In the above figure, z axis intersects projection plane whereas x and y axis remain parallel to projection plane.

- **Two Point Perspective Projection –**

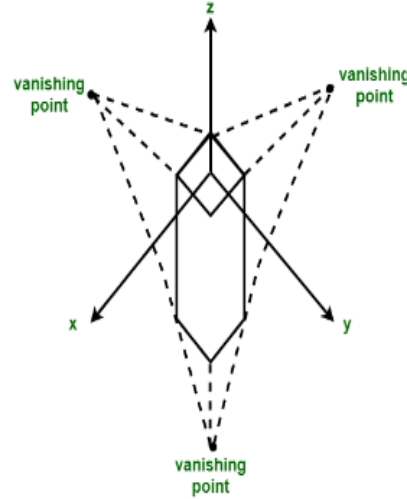
Two point perspective projection occurs when projection plane intersects two of principal axis.



In the above figure, projection plane intersects x and y axis whereas z axis remains parallel to projection plane.

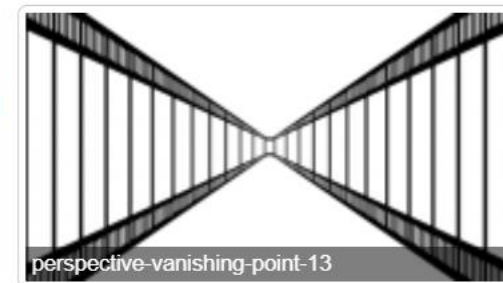
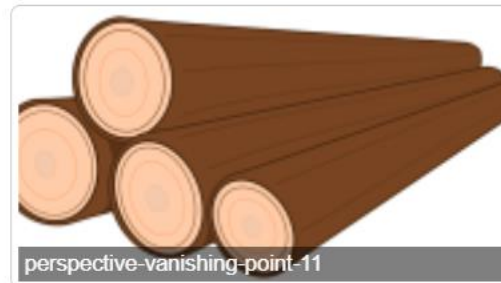
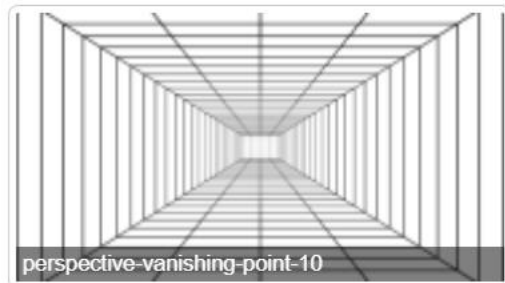
- **Three Point Perspective Projection -**

Three point perspective projection occurs when all three axis intersects with projection plane. There is no any principle axis which is parallel to projection plane.

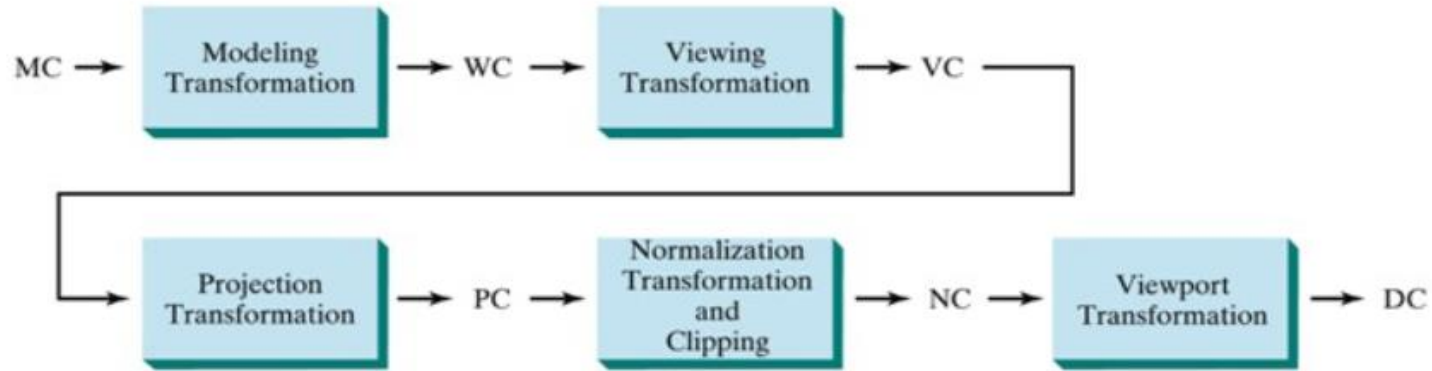


Vanishing Point

Perspective projection produces realistic views but it doesn't preserve relative proportions of object dimensions. Projection of distant objects is smaller than the projection of objects of the same size that after projection certain sets of parallel lines appear to meet at some point on the projection plane. These points are called **Vanishing Point**.



General 3D Viewing Pipeline



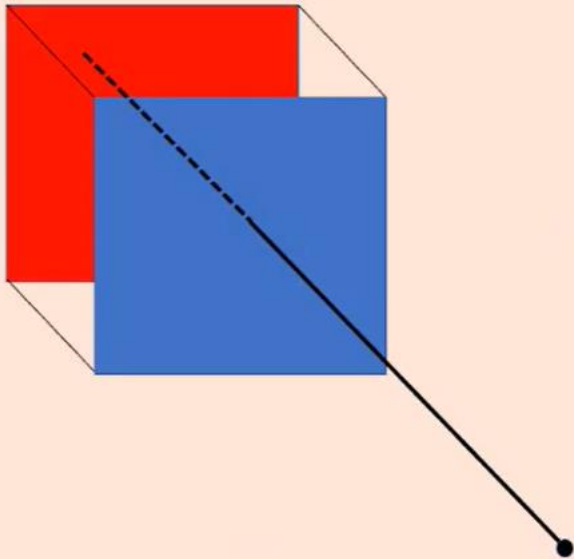
- Modeling coordinates (MC)
- World coordinates (WC)
- Viewing coordinates (VC)
- Projection coordinates (PC)
- Normalized coordinates (NC)
- Device coordinates (DC)

- Once the scene has been modeled, world-coordinate positions are converted to viewing coordinates.
- The viewing-coordinate system is used in graphics packages as a reference for specifying the observer viewing position and the position of the projection plane, which we can think of in analogy with the camera film plane.
- Next, projection operations are performed to convert the viewing-coordinate description of the scene to coordinate positions on the projection plane, which will then be mapped to the output device.
- Objects outside the specified viewing limits are clipped from further consideration, and the remaining objects are processed through visible-surface identification and surface-rendering procedures to produce the display within the device viewport.

Visible Surface Detection

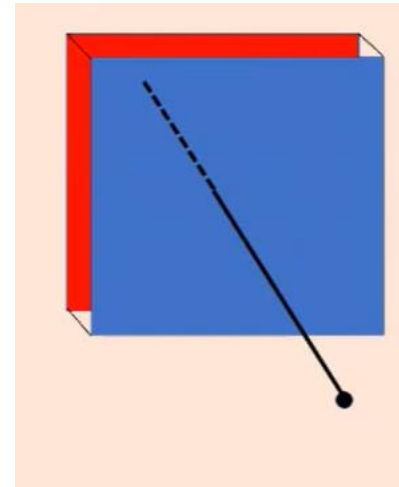
- To identify those parts of a scene that are visible from a chosen viewing position.
- Surfaces which are obscured by other opaque surfaces along the line of sight (projection) are invisible to the viewer.
- Classification:
 - Object space methods - Physical coordinate system
 - Image space methods - Screen coordinate system

Visible & Not-Visible



Back Face

Front Face



Visible-Surface Detection Algorithms

Hidden-Surface Elimination Algorithms

Object-Space Methods

Compares Objects & Parts of Objects

Image-Space Methods

Compares Point by Point at each Pixel Position at Projection Plane

1. Object-space Methods

- Compare objects and parts of objects to each other within the scene definition to determine which surfaces, as a whole, we should label as visible:

- For each object in the scene do

Begin

- 1. Determine those parts of the object whose view is unobstructed by other parts of it or any other object with respect to the viewing specification.
- 2. Draw those parts in the object color.

End

2. Image-space Methods

- Visibility is determined point by point at each pixel position on the projection plane.
- For each pixel in the image do

Begin

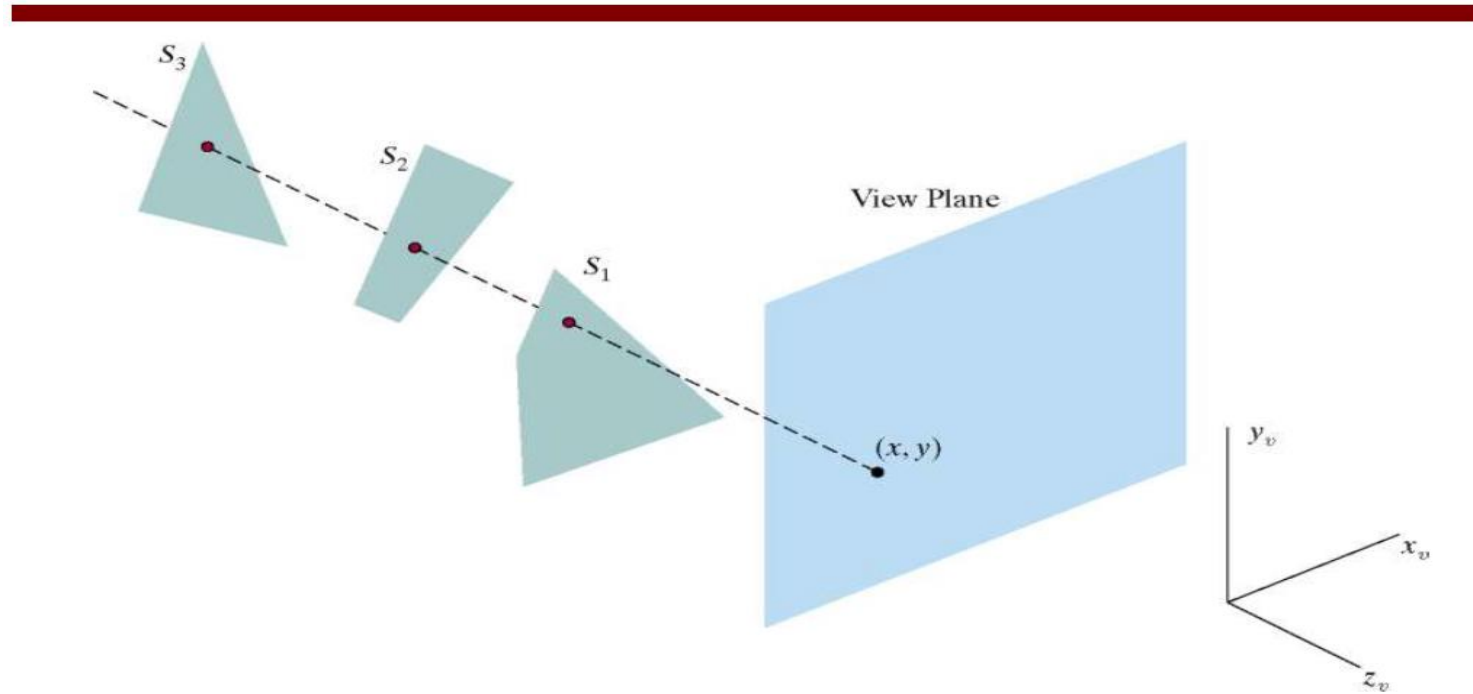
- 1. Determine the object closest to the viewer that is pierced by the projector through the pixel
- 2. Draw the pixel in the object colour.

End

Depth-Buffer Method

- Also known as z-buffer method.
- It is an image space approach
 - Each surface is processed separately one pixel position at a time across the surface
 - The depth values for a pixel are compared and the closest (smallest z) surface determines the color to be displayed in the frame buffer.
 - Applied very efficiently on polygon surfaces
 - Surfaces are processed in any order

Depth-Buffer Method



Buffers used

Two buffer areas are required

- Depth buffer
 - Store depth values for each (x,y) position/Z value
 - All positions are initialized to minimum depth
- Refresh buffer
 - Stores the intensity values for each position
 - All positions are initialized to the background intensity

Algorithm

Step-1 – Set the buffer values –

Depthbuffer $x, y = 0$

Framebuffer $x, y = \text{background color}$

Step-2 – Process each polygon *One at a time*

For each projected x, y pixel position of a polygon, calculate depth z .

If $Z > \text{depthbuffer } x, y$

 Compute surface color,

 set depthbuffer $x, y = z$,

 framebuffer $x, y = \text{surfacecolor } x, y$

After all Surfaces have been processed

- Depth buffer contains Depth Values for the Visible Surfaces
- Frame buffer contains Corresponding Color Values for those Surfaces

- Two buffer areas are required. A **depth buffer** is used to store **depth values for each (x, y)** position as surfaces are processed, and the **refresh buffer** stores the **intensity values** for each position.
- Initially, all positions in the depth buffer are set to 0 (minimum depth), and the refresh buffer is initialized to the background intensity.
- Each surface listed in the polygon tables is then processed, one scan line at a time, calculating the depth (z value) at each (x, y) pixel position. The calculated depth is compared to the value previously stored in the depth buffer at that position.
- If the calculated depth is greater than the value stored in the depth buffer, the new depth value is stored and the surface intensity at that position is determined and in the same xy location in the refresh buffer.

Depth values for a surface position (x, y) are calculated from the plane equation for each surface

$$Ax + By + Cz + D = 0$$

$$z = \frac{-Ax - By - D}{C}$$

- For any scan line, adjacent horizontal positions across the line differ by 1, and a vertical y value on an adjacent scan line differs by 1.
- If the depth of position (x, y) has been determined to be z , then the depth of next position $(x+1, y)$ along the scan line

$$z' = \frac{-A(x+1) - By - D}{C}$$

$$= \frac{-Ax - By - D - A}{C}$$

$$= z - A/C$$

$$\boxed{z' = z - \frac{A}{C}}$$

$\frac{A}{C}$ constant for each surface.

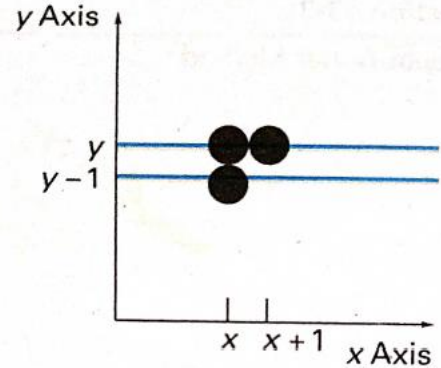


Figure 13-5

From position (x, y) on a scan line, the next position across the line has coordinates $(x+1, y)$, and the position immediately below on the next line has coordinates $(x, y-1)$.

To calculate depth values down the edge

$$\frac{y-y'}{x-x'} = m$$

$$\frac{y-(y-1)}{x-x'} = m$$

$$x-x' = \frac{1}{m}$$

$$\boxed{x' = x - \frac{1}{m}}$$

$$\begin{aligned}
 Z &= \frac{-Ax - By - D}{C} \\
 Z_{y+1} &= \frac{-A(x - x_m) - B(y - 1) - D}{C} \\
 &= \frac{-Ax + A/m - By + B - D}{C} \\
 &= \frac{-Ax - By - D}{C} + \frac{A/m + B}{C}
 \end{aligned}$$

$$\boxed{Z' = Z + \frac{A/m + B}{C}}$$

If we are processing down a vertical edge, the slope is infinite and equation becomes

$$\boxed{Z' = Z + \frac{B}{C}}$$

Scan line method

- This image space method for removing hidden surfaces is an extension of the scan-line algorithm.
- It deals with multiple surfaces.
- As each scan line is processed, all polygon surfaces intersecting that line are examined to determine which are visible.
- Across each scan line, depth calculations are made for each overlapping surface to determine which is nearest to the view plane.
- When the visible surface has been determined, the intensity value for that position is entered into the refresh buffer.

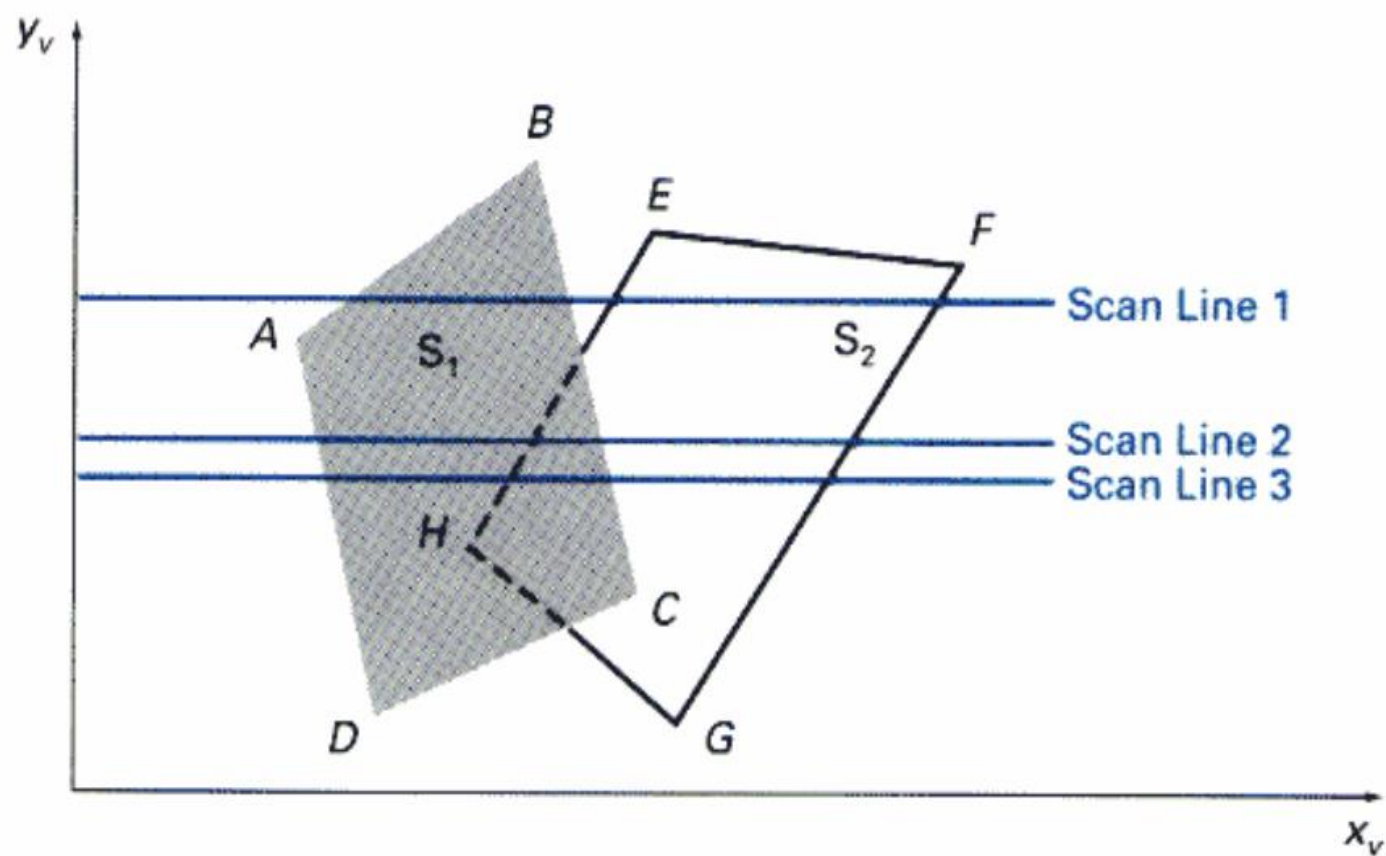
- Here two tables are used, an edge table and a polygon table.
- The edge table contains information for all lines(edges) intersecting the scan line. For each scan line, there exists an entry which contains coordinate endpoints for each edge intersecting the line, the inverse slope of each line, and pointers into the polygon table to identify the surfaces bounded by each line.
- The polygon table contains coefficients of the plane equation for each surface, intensity information for the surfaces, and possibly pointers into the edge table

- To facilitate the search for surfaces crossing the scan line, set up an active list of edges from information in the edge table. **This active list** will contain only edges that cross the current scan line, sorted in order of increasing x .
- In addition, define a flag for each surface that is set on or off.
- Scan lines are processed from left to right.
- At the leftmost boundary of a surface, the surface flag is turned on; and at the rightmost boundary, it is turned off

- 3 tables
- Edge table
 - Coordinate endpoints for each line
 - Slope of each line
 - Pointers into the polygon table
 - Identify the surfaces bounded by each line
- Polygon table
 - Coefficients of the plane equation for each surface
 - Intensity information for the surfaces
 - Pointers into the edge table
- Active list
 - Contain only edges across the current scan line
 - Sorted in order of increasing x

Flag for each surface

- Indicate whether inside or outside of the surface
 - At the leftmost boundary of a surface
 - The surface flag is turned on
 - At the rightmost boundary of a surface
 - The surface flag is turned off



Scan lines crossing the projection of two surfaces, S_1 and S_2 , in the view plane. Dashed lines indicate the boundaries of hidden surfaces.

- Active list for scan line 1

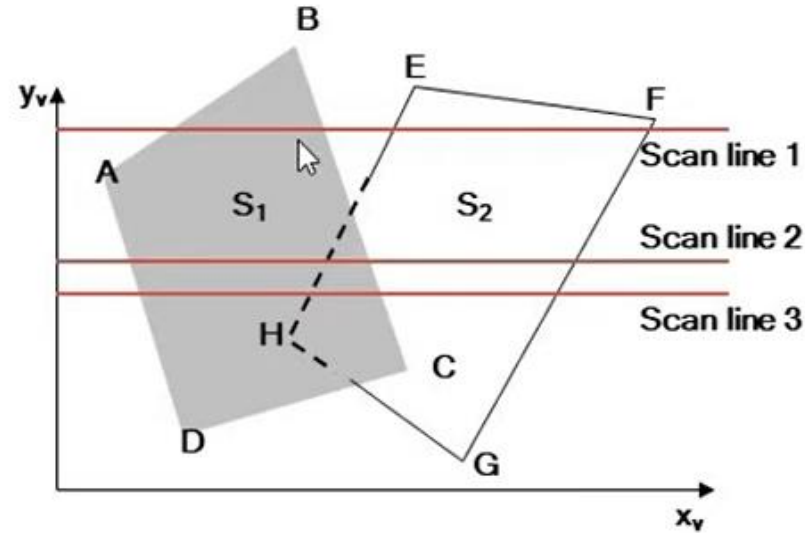
- Edge table

- AB, BC, EH, and FG

- Between AB and BC, only the flag for surface S_1 is on

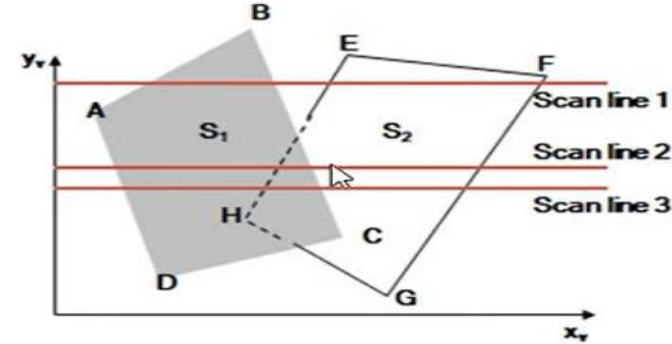
- No depth calculations are necessary
 - Intensity for surface S_1 is entered into the refresh buffer

- Similarly, between EH and FG, only the flag for S_2 is on



For scan line 2

- AD, EH, BC, and FG
 - Between AD and EH, only the flag for S_1 is on
 - Between EH and BC, the flags for both surfaces are on
 - Depth calculation is needed using plane coefficient.
 - If depth $S_1 < S_2$ Intensities for S_1 are loaded into the refresh buffer until BC
 - After that S_1 intensity of surface S_2 is stored until FG
- Scan line 3 has the same active list as scan line 2
 - Unnecessary to make depth calculations between EH and BC



Advantages

- Any number of overlapping surfaces can be processed , Flags for the surfaces are set to indicate whether a position is inside or outside, and depth calculations are performed when surfaces overlap
- Take advantage of coherence , When these coherence methods are used, we need to be careful to keep track of which surface section is visible on each scan line.
- This works only if surfaces do not cut through or otherwise cyclically overlap each other

disadvantages

- It does not work for the surface that cut through or cyclic overlap each other

- If any kind of cyclic overlap is present in a scene, we can divide the surfaces to eliminate the overlaps.
- The dashed lines in this figure indicate where planes could be sub divided to form two distinct surfaces, so that the cyclic overlaps are eliminated.

